

**LAPORAN TUGAS 7**

**SISTEM OPERASI**



**Dosen Pengampu:**

**Triawan Adi Cahyanto, M.Kom**

**Disusun Oleh:**

**Fagil Lola Prihernando (2410651031)**

**PRODI TEKNIK INFORMATIKA**

**FAKULTAS TEKNIK**

**UNIVERSITAS MUHAMMADIYAH JEMBER**

**2025**

## DAFTAR ISI

|                                       |           |
|---------------------------------------|-----------|
| <b>BAB 1 .....</b>                    | <b>4</b>  |
| <b>PENDAHULUAN .....</b>              | <b>4</b>  |
| <b>1.1 LATAR BELAKANG .....</b>       | <b>4</b>  |
| <b>1.2 TUJUAN PRAKTIKUM .....</b>     | <b>4</b>  |
| <b>1.3 SPESIFIKASI SISTEM.....</b>    | <b>5</b>  |
| <b>BAB II .....</b>                   | <b>6</b>  |
| <b>DASAR TEORI.....</b>               | <b>6</b>  |
| <b>2.1 MEMORY MANAGEMENT.....</b>     | <b>6</b>  |
| <b>2.2 CONTIGUOUS ALLOCATION.....</b> | <b>6</b>  |
| <b>2.3 PAGING.....</b>                | <b>6</b>  |
| <b>2.4 PAGE TABLE STRUCTURES.....</b> | <b>6</b>  |
| <b>2.5 SWAPPING .....</b>             | <b>7</b>  |
| <b>2.6 ARCHITECTURE .....</b>         | <b>7</b>  |
| <b>BAB III.....</b>                   | <b>8</b>  |
| <b>JAWABAN TUGAS PRAKTIKUM .....</b>  | <b>8</b>  |
| <b>3.1 TUGAS 1.....</b>               | <b>8</b>  |
| <b>3.2 TUGAS 2.....</b>               | <b>13</b> |
| <b>3.3 TUGAS 3.....</b>               | <b>22</b> |
| <b>3.4 TUGAS 4.....</b>               | <b>47</b> |
| <b>3.5 TUGAS 5.....</b>               | <b>54</b> |
| <b>3.6 TUGAS 6.....</b>               | <b>62</b> |
| <b>BAB IV .....</b>                   | <b>66</b> |
| <b>ANALISIS DAN PEMBAHASAN.....</b>   | <b>66</b> |
| <b>4.1 ANALISIS TUGAS 1 .....</b>     | <b>66</b> |
| <b>4.2 ANALISIS TUGAS 2 .....</b>     | <b>70</b> |
| <b>4.3 ANALISIS TUGAS 3 .....</b>     | <b>71</b> |

|                                   |           |
|-----------------------------------|-----------|
| <b>4.4 ANALISIS TUGAS 4 .....</b> | <b>75</b> |
| <b>4.5 ANALISIS TUGAS 5 .....</b> | <b>77</b> |
| <b>4.6 ANALISIS TUGAS 6 .....</b> | <b>78</b> |
| <b>BAB V .....</b>                | <b>80</b> |
| <b>KESIMPULAN DAN SARAN .....</b> | <b>80</b> |
| <b>DAFTAR PUSTAKA .....</b>       | <b>81</b> |

# **BAB 1**

## **PENDAHULUAN**

### **1.1 LATAR BELAKANG**

Manajemen memori merupakan salah satu komponen paling penting dalam sistem operasi karena berperan langsung dalam mengatur penggunaan sumber daya utama komputer, yaitu memori. Tanpa pengelolaan memori yang baik, sistem tidak mampu menjalankan banyak proses secara efisien, stabil, dan aman. Perkembangan teknologi komputasi modern menuntut sistem operasi untuk mendukung berbagai mekanisme manajemen memori seperti alokasi dinamis, paging, multi-level page table, swapping, serta optimasi berbasis arsitektur perangkat keras.

Melalui modul praktikum ini, mahasiswa diajak untuk memahami konsep manajemen memori tidak hanya secara teoritis, tetapi juga melalui implementasi langsung menggunakan bahasa pemrograman C dan pengamatan pada sistem nyata. Praktikum mencakup analisis memory layout proses, implementasi algoritma alokasi memori, simulasi sistem paging lanjutan, perbandingan struktur page table, mekanisme swapping, hingga analisis arsitektur memori pada prosesor modern seperti Intel dan ARM.

Dengan melakukan simulasi, benchmarking, dan analisis performa, mahasiswa dapat memahami trade-off yang terjadi pada setiap pendekatan manajemen memori. Praktikum ini diharapkan mampu meningkatkan pemahaman konseptual sekaligus keterampilan praktis mahasiswa dalam menganalisis dan merancang sistem manajemen memori yang efisien dan sesuai dengan kebutuhan sistem operasi modern.

### **1.2 TUJUAN PRAKTIKUM**

1. Memahami konsep dan mekanisme manajemen memori sistem operasi melalui implementasi dan simulasi langsung.
2. Menganalisis performa berbagai algoritma manajemen memori seperti alokasi kontigu, paging, page replacement, dan swapping.
3. Membandingkan struktur dan karakteristik arsitektur memori pada sistem nyata serta hubungannya dengan teori.

### **1.3 SPESIFIKASI SISTEM**

1. Sistem Operasi: Linux (Ubuntu)
2. Bahasa Pemrograman: C (GNU Compiler Collection / GCC)
3. Arsitektur Prosesor: 64-bit (x86-64 atau ARMv8)

## **BAB II**

### **DASAR TEORI**

#### **2.1 MEMORY MANAGEMENT**

Manajemen memori merupakan salah satu fungsi utama sistem operasi yang bertugas mengatur penggunaan memori utama agar dapat dimanfaatkan secara efisien oleh berbagai proses. Sistem operasi harus menentukan bagian memori yang digunakan, bagian yang masih kosong, serta proses mana yang berhak mengakses memori tertentu. Manajemen memori juga bertanggung jawab dalam alokasi dan dealokasi memori, translasi alamat logis ke alamat fisik, serta perlindungan memori antar proses. Dengan pengelolaan memori yang baik, sistem dapat menjalankan banyak proses secara bersamaan tanpa konflik, meningkatkan kinerja, serta mencegah kesalahan seperti akses memori ilegal.

#### **2.2 CONTIGUOUS ALLOCATION**

Contiguous allocation adalah metode alokasi memori di mana setiap proses ditempatkan pada satu blok memori yang berurutan secara fisik. Metode ini relatif sederhana untuk diimplementasikan dan memiliki kecepatan akses yang tinggi karena alamat memori bersifat linear. Namun, contiguous allocation memiliki kelemahan utama berupa fragmentasi, baik internal maupun eksternal. Untuk menentukan blok memori yang akan digunakan, sistem dapat menerapkan algoritma seperti First-Fit, Best-Fit, dan Worst-Fit. Meskipun efisien untuk sistem sederhana, metode ini kurang cocok untuk sistem modern dengan kebutuhan alokasi memori yang dinamis.

#### **2.3 PAGING**

Paging merupakan teknik manajemen memori yang membagi memori logis dan memori fisik menjadi unit-unit berukuran tetap yang disebut page dan frame. Dengan paging, proses tidak perlu ditempatkan secara berurutan di memori fisik, sehingga masalah fragmentasi eksternal dapat dihilangkan. Sistem operasi menggunakan page table untuk memetakan halaman logis ke frame fisik. Paging juga mendukung implementasi memori virtual, yang memungkinkan sistem menjalankan program dengan ukuran lebih besar dari kapasitas memori fisik. Meskipun efektif, paging menimbulkan overhead tambahan akibat penggunaan page table dan translasi alamat.

#### **2.4 PAGE TABLE STRUCTURES**

Struktur page table dikembangkan untuk mengatasi keterbatasan ukuran page table tunggal, terutama pada sistem dengan ruang alamat besar. Salah satu pendekatan yang umum digunakan adalah multi-level page table, seperti two-level atau four-level page table. Pada struktur ini, page table dibagi menjadi beberapa tingkat sehingga tabel hanya dialokasikan ketika dibutuhkan. Hal ini dapat menghemat memori secara signifikan jika hanya sebagian ruang alamat virtual yang digunakan.

Namun, struktur bertingkat juga menambah kompleksitas translasi alamat dan dapat meningkatkan waktu akses memori karena harus melewati beberapa level tabel.

## **2.5 SWAPPING**

Swapping adalah teknik manajemen memori yang memindahkan proses dari memori utama ke ruang penyimpanan sekunder (swap space) ketika memori fisik tidak mencukupi. Dengan swapping, sistem dapat menjalankan lebih banyak proses dibandingkan kapasitas RAM yang tersedia. Sistem operasi menggunakan algoritma tertentu, seperti FIFO, LRU, atau Priority-based, untuk menentukan proses mana yang akan dipindahkan. Meskipun membantu meningkatkan utilisasi memori, swapping memiliki kelemahan berupa overhead waktu yang besar akibat operasi I/O disk, sehingga penggunaannya biasanya dibatasi untuk kondisi tertentu.

## **2.6 ARCHITECTURE**

Arsitektur komputer memiliki pengaruh besar terhadap desain dan implementasi manajemen memori. Perbedaan arsitektur, seperti IA-32, x86-64, dan ARMv8, menentukan ukuran alamat, struktur paging, serta jumlah level page table yang digunakan. Misalnya, arsitektur 64-bit mendukung ruang alamat yang jauh lebih besar dan menggunakan multi-level paging untuk efisiensi. Beberapa arsitektur juga mendukung berbagai ukuran halaman untuk menyesuaikan kebutuhan aplikasi. Pemahaman arsitektur sangat penting dalam memahami cara sistem operasi mengelola memori secara optimal dan memanfaatkan fitur perangkat keras.

## BAB III

### JAWABAN TUGAS PRAKTIKUM

#### 3.1 TUGAS 1

##### 3.1.1 Deskripsi Tugas

Tugas 1 bertujuan untuk menganalisis memory layout suatu program C secara lebih mendalam dengan memodifikasi program `address_demo.c`. Pada tugas ini, mahasiswa diminta menambahkan berbagai fitur seperti alokasi array statis dan dinamis untuk mengamati perbedaan alamat memori, pembuatan beberapa fungsi guna membandingkan alamat pada text segment, serta penggunaan fungsi rekursif untuk melihat perubahan alamat stack di setiap level pemanggilan. Selain itu, alamat string literals juga ditampilkan untuk mengidentifikasi letaknya dalam memori. Hasil eksekusi dianalisis dengan menghitung jarak antar segmen memori, mengamati pengaruh ASLR melalui eksekusi berulang, serta membandingkan alamat yang diperoleh dengan memory map pada `/proc/[pid]/maps`.

##### 3.1.2 Source Code

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

/* prototype */
int main();

/* variabel global */
int global_var = 100;
int static_array[3] = {10, 20, 30};
const char *text = "Main Memory Praktikum";

/* fungsi biasa */
void func1() {}
void func2() {}
void func3() {}

/* fungsi rekursif */
void recursive(int level) {
    int x = level;
```



```

printf(" Level %d local_var address: %p value: %d\n",
       level, (void*)&x, x);

if (level < 2)
    recursive(level + 1);
}

void print_memory() {
    int local = 5;
    int *heap1 = malloc(sizeof(int));
    int *heap2 = malloc(sizeof(int));
    int *dyn_array = malloc(3 * sizeof(int));

    /* isi nilai */
    *heap1 = 50;
    *heap2 = 75;
    dyn_array[0] = 1;

    printf("\n==== ENHANCED MEMORY LAYOUT ====\n\n");

    printf("1. Text Segment:\n");
    printf("  main() : %p\n", (void*)main);
    printf("  func1() : %p\n", (void*)func1);
    printf("  func2() : %p\n", (void*)func2);
    printf("  func3() : %p\n\n", (void*)func3);

    printf("2. Data Segment:\n");
    printf("  global_var address: %p value: %d\n",
           (void*)&global_var, global_var);
    printf("  static_array[0] address: %p value: %d\n",
           (void*)&static_array[0], static_array[0]);
    printf("  string literal address: %p value: %s\n\n",
           (void*)text, text);

    printf("3. Heap:\n");
    printf("  heap1 address: %p value: %d\n",

```

```

        (void*)heap1, *heap1);
    printf("  heap2 address: %p value: %d\n",
        (void*)heap2, *heap2);
    printf("  dyn_array[0] address: %p value: %d\n\n",
        (void*)&dyn_array[0], dyn_array[0]);

    printf("4. Stack:\n");
    printf("  local address: %p value: %d\n\n",
        (void*)&local, local);

    printf("4. Stack (Recursive Calls):\n");
    recursive(0);

    free(heap1);
    free(heap2);
    free(dyn_array);
}

int main() {
    printf("PID: %d\n", getpid());
    print_memory();

    printf("\nTekan Enter untuk melihat memory map...");
    getchar();

    char cmd[100];
    sprintf(cmd, "cat /proc/%d/maps", getpid());
    system(cmd);

    return 0;
}

```

### 3.1.3 Screenshot Output

```
asus@LAPTOP-VRS206DG:~/praktikum-memory/background$ nano address_demo.c
asus@LAPTOP-VRS206DG:~/praktikum-memory/background$ gcc -o address_demo address_demo.c
asus@LAPTOP-VRS206DG:~/praktikum-memory/background$ ./address_demo
PID: 7992

=== ENHANCED MEMORY LAYOUT ===

1. Text Segment:
   main() : 0x6281951ef5b5
   func1() : 0x6281951ef249
   func2() : 0x6281951ef254
   func3() : 0x6281951ef25f

2. Data Segment:
   global_var address: 0x6281951f2010 value: 100
   static_array[0] address: 0x6281951f2018 value: 10
   string literal address: 0x6281951f0008 value: Main Memory Praktikum

3. Heap:
   heap1 address: 0x6281c4b756b0 value: 50
   heap2 address: 0x6281c4b756d0 value: 75
   dyn_array[0] address: 0x6281c4b756f0 value: 1

4. Stack:
   Local address: 0x7ffdd046c87c value: 5

4. Stack (Recursive Calls):
   Level 0 local_var address: 0x7ffdd046c854 value: 0
   Level 1 local_var address: 0x7ffdd046c824 value: 1
   Level 2 local_var address: 0x7ffdd046c7f4 value: 2

Tekan Enter untuk melihat memory map...
6281951ef000-6281951ef000 r-p 00000000 00:30 21064 /home/asus/praktikum-memory/background/address_demo
6281951ef000-6281951ef000 r-xp 00001000 00:30 21064 /home/asus/praktikum-memory/background/address_demo
6281951f0000-6281951f1000 r-p 00002000 00:30 21064 /home/asus/praktikum-memory/background/address_demo
6281951f1000-6281951f2000 r-p 00002000 00:30 21064 /home/asus/praktikum-memory/background/address_demo
6281951f2000-6281951f3000 rw-p 00003000 00:30 21064 /home/asus/praktikum-memory/background/address_demo
6281c4b75000-6281c4b75600 rw-p 00000000 00:00 0 [heap]
7c9176000000-7c9176020000 r-p 00000000 00:30 46701 /usr/lib/x86_64-linux-gnu/libc.so.6
7c9176020000-7c91761b0000 r-xp 00020000 00:30 46701 /usr/lib/x86_64-linux-gnu/libc.so.6
7c91761b0000-7c91761ff000 r-p 001b0000 00:30 46701 /usr/lib/x86_64-linux-gnu/libc.so.6
7c91761ff000-7c9176203000 r-p 001fe000 00:30 46701 /usr/lib/x86_64-linux-gnu/libc.so.6
7c9176203000-7c9176205000 rw-p 00202000 00:30 46701 /usr/lib/x86_64-linux-gnu/libc.so.6
7c9176205000-7c9176212000 rw-p 00000000 00:00 0
7c9176313000-7c9176316000 rw-p 00000000 00:00 0
7c9176316000-7c917631d000 r-xp 00000000 00:00 0
7c917631d000-7c917634d000 r-p 00000000 00:30 46698 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7c917634d000-7c917634e000 r-p 00000000 00:30 46698 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7c917634e000-7c9176379000 r-xp 00001000 00:30 46698 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7c9176379000-7c9176383000 r-p 0002c000 00:30 46698 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7c9176383000-7c9176385000 r-p 00036000 00:30 46698 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7c9176385000-7c9176387000 rw-p 00038000 00:30 46698 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7ffdd0400000-7ffdd0406f000 rw-p 00000000 00:00 0 [stack]
7ffdd054e000-7ffdd0552000 r-p 00000000 00:00 0 [vvar]
7ffdd0552000-7ffdd0554000 r-xp 00000000 00:00 0 [vdso]
asus@LAPTOP-VRS206DG:~/praktikum-memory/background$ |
```

### 3.1.4 Penjelasan Implementasi

#### Proses Kompilasi Program

- Preprocessing
  - Direktif `#include <stdio.h>`, `#include <stdlib.h>`, dan `#include <unistd.h>` diproses oleh preprocessor.
  - Deklarasi fungsi, variabel global, string literal, dan konstanta disiapkan sebelum kompilasi.
  - Tidak terdapat makro khusus, sehingga tahap ini berfokus pada penyertaan header.
- Kompilasi
  - Kode sumber C diterjemahkan menjadi kode assembly.
  - Struktur program seperti fungsi `main()`, `print_memory()`, dan `recursive()` dianalisis.
  - Variabel global (`global_var`, `static_array`, `text`) ditempatkan ke segmen data.
  - Fungsi-fungsi (`main`, `func1`, `func2`, `func3`, `recursive`) dipetakan ke text segment.
- Assembly
  - Kode assembly dikonversi menjadi file objek berisi instruksi mesin.
  - Referensi simbol fungsi dan variabel disimpan untuk proses linking.

- Linking
  - File objek digabungkan dengan library standar C.
  - Fungsi seperti printf(), malloc(), free(), getpid(), dan system() dihubungkan.
  - Berkas executable akhir dihasilkan.

## Proses Eksekusi Program

- Loading Progra
  - Sistem operasi memuat program ke dalam memori.
  - Text segment berisi kode mesin fungsi-fungsi program.
  - Data segment memuat variabel global dan string literal.
  - Heap dan stack disiapkan untuk alokasi dinamis dan pemanggilan fungsi.
- Eksekusi Awal
  - Fungsi main() dijalankan sebagai titik awal program.
  - Program mencetak Process ID (PID) menggunakan getpid().
- Pemanggilan print\_memory()
  - Variabel lokal local dialokasikan di stack.
  - Alokasi memori dinamis dilakukan menggunakan malloc() untuk heap1, heap2, dan dyn\_array.
  - Nilai awal diberikan ke variabel heap dan array dinamis.
- Penampilan Layout Memori
  - Alamat fungsi ditampilkan sebagai representasi text segment.
  - Alamat dan nilai variabel global ditampilkan sebagai data segment.
  - Alamat hasil malloc() ditampilkan sebagai heap.
  - Alamat variabel lokal ditampilkan sebagai stack.
- Pemanggilan Fungsi Rekursif
  - Fungsi recursive() dipanggil berulang.
  - Setiap pemanggilan membuat stack frame baru.
  - Alamat variabel lokal berbeda pada setiap level rekursi ditampilkan.
- Dealokasi Memori
  - Memori heap dibebaskan menggunakan free() untuk mencegah memory leak.
  - Menampilkan Memory Map
  - Program menunggu input pengguna (getchar()).
  - File /proc/[PID]/maps ditampilkan menggunakan perintah sistem.

- Layout memori proses ditampilkan secara nyata oleh kernel Linux.
- Terminasi Program
  - Program selesai dieksekusi.
  - Sistem operasi membersihkan seluruh resource dan stack frame.
  - Program keluar dengan status 0.

## 3.2 TUGAS 2

### 3.2.1 Deskripsi Tugas

Tugas 2 bertujuan mengembangkan program memory allocator dengan menambahkan algoritma Next-Fit serta melakukan analisis performa berbagai algoritma alokasi memori. Mahasiswa diminta mengimplementasikan Next-Fit yang bekerja seperti First-Fit, namun memulai pencarian dari posisi alokasi terakhir. Selain itu, ditambahkan fitur untuk menghitung fragmentasi memori guna mengevaluasi efisiensi penggunaan ruang. Program kemudian diuji melalui benchmark testing untuk membandingkan First-Fit, Best-Fit, Worst-Fit, dan Next-Fit berdasarkan waktu eksekusi, tingkat fragmentasi, dan kegagalan alokasi. Hasil pengujian dianalisis dalam bentuk tabel dan grafik untuk menentukan algoritma paling cepat, paling efisien, serta kondisi penggunaan terbaik bagi masing-masing algoritma.

### 3.2.2 Source Code

```
#include <stdio.h>
#include <time.h>
#define MAX_BLOCKS 100

typedef struct {
    int size;
    int free;    // 1 = free, 0 = allocated
    int process_id;
} Block;

Block memory[MAX_BLOCKS];
int block_count = 1;
int last_alloc_index = 0;

/* ===== INIT MEMORY ===== */
```

```

void init_memory(int total_size) {
    memory[0].size = total_size;
    memory[0].free = 1;
    memory[0].process_id = -1;
    block_count = 1;
    last_alloc_index = 0;
}

/* ===== DISPLAY ===== */
void display_memory() {
    printf("\nMemory State:\n");
    for (int i = 0; i < block_count; i++) {
        if (memory[i].free)
            printf("Block %d: FREE | Size = %d\n", i, memory[i].size);
        else
            printf("Block %d: P%d | Size = %d\n", i, memory[i].process_id, memory[i].size);
    }
}

/* ===== DEALLOCATE ===== */
void deallocate(int pid) {
    for (int i = 0; i < block_count; i++) {
        if (!memory[i].free && memory[i].process_id == pid) {
            memory[i].free = 1;
            memory[i].process_id = -1;
            return;
        }
    }
}

/* ===== FIRST FIT ===== */
int first_fit(int pid, int size) {
    for (int i = 0; i < block_count; i++) {
        if (memory[i].free && memory[i].size >= size) {

            if (memory[i].size > size) {

```

```

        for (int j = block_count; j > i + 1; j--)
            memory[j] = memory[j - 1];

        memory[i + 1].size = memory[i].size - size;
        memory[i + 1].free = 1;
        memory[i + 1].process_id = -1;
        block_count++;
    }

    memory[i].size = size;
    memory[i].free = 0;
    memory[i].process_id = pid;
    return 1;
}
}
return 0;
}

/* ===== BEST FIT ===== */
int best_fit(int pid, int size) {
    int best = -1;

    for (int i = 0; i < block_count; i++) {
        if (memory[i].free && memory[i].size >= size) {
            if (best == -1 || memory[i].size < memory[best].size)
                best = i;
        }
    }

    if (best == -1) return 0;

    if (memory[best].size > size) {
        for (int j = block_count; j > best + 1; j--)
            memory[j] = memory[j - 1];

        memory[best + 1].size = memory[best].size - size;
    }
}

```

```

    memory[best + 1].free = 1;
    memory[best + 1].process_id = -1;
    block_count++;
}

memory[best].size = size;
memory[best].free = 0;
memory[best].process_id = pid;
return 1;
}

/* ===== WORST FIT ===== */
int worst_fit(int pid, int size) {
    int worst = -1;

    for (int i = 0; i < block_count; i++) {
        if (memory[i].free && memory[i].size >= size) {
            if (worst == -1 || memory[i].size > memory[worst].size)
                worst = i;
        }
    }

    if (worst == -1) return 0;

    if (memory[worst].size > size) {
        for (int j = block_count; j > worst + 1; j--)
            memory[j] = memory[j - 1];

        memory[worst + 1].size = memory[worst].size - size;
        memory[worst + 1].free = 1;
        memory[worst + 1].process_id = -1;
        block_count++;
    }

    memory[worst].size = size;
    memory[worst].free = 0;

```



```

memory[worst].process_id = pid;
return 1;
}

/* ===== NEXT FIT ===== */
int next_fit(int pid, int size) {
    int i = last_alloc_index;
    int scanned = 0;

    while (scanned < block_count) {
        if (memory[i].free && memory[i].size >= size) {

            if (memory[i].size > size) {
                for (int j = block_count; j > i + 1; j--)
                    memory[j] = memory[j - 1];

                memory[i + 1].size = memory[i].size - size;
                memory[i + 1].free = 1;
                memory[i + 1].process_id = -1;
                block_count++;
            }

            memory[i].size = size;
            memory[i].free = 0;
            memory[i].process_id = pid;

            last_alloc_index = i;
            return 1;
        }

        i = (i + 1) % block_count;
        scanned++;
    }
    return 0;
}

```

```
/* ===== FRAGMENTATION ===== */
```

```
void calculate_fragmentation() {  
    int total_free = 0, largest = 0, holes = 0;  
  
    for (int i = 0; i < block_count; i++) {  
        if (memory[i].free) {  
            total_free += memory[i].size;  
            holes++;  
            if (memory[i].size > largest)  
                largest = memory[i].size;  
        }  
    }  
  
    float ext = (total_free > 0) ?  
        (1.0 - (float)largest / total_free) * 100 : 0;  
  
    printf("Total Free Space    : %d\n", total_free);  
    printf("Largest Free Block    : %d\n", largest);  
    printf("Free Holes              : %d\n", holes);  
    printf("External Fragmentation: %.2f%%\n", ext);  
}
```

```
/* ===== BENCHMARK ===== */
```

```
void benchmark(const char *name, int (*alloc)(int,int)) {  
    init_memory(1000);  
    clock_t start = clock();  
  
    alloc(1,100);  
    alloc(2,200);  
    alloc(3,300);  
    alloc(4,250);  
    alloc(5,150);  
  
    deallocate(2);  
    deallocate(4);  
}
```

```
    alloc(6,180);
    alloc(7,220);
    alloc(8,100);

    clock_t end = clock();

    printf("\n=== %s ===\n", name);
    printf("Execution Time: %.2f us\n",
        (double)(end-start)*1e6/CLOCKS_PER_SEC);

    calculate_fragmentation();
    display_memory();
}

/* ===== MAIN ===== */
int main() {
    benchmark("First Fit", first_fit);
    benchmark("Best Fit", best_fit);
    benchmark("Worst Fit", worst_fit);
    benchmark("Next Fit", next_fit);
    return 0;
}
```

### 3.2.3 Screenshot Output

```
asus@LAPTOP-VRS296DG:~/praktikum-memory/contiguous-allocation$ ./memory_allocator

=== First Fit ===
Execution Time: 3.00 us
Total Free Space : 50
Largest Free Block : 30
Free Holes : 2
External Fragmentation: 40.00%

Memory State:
Block 0: P1 | Size = 100
Block 1: P6 | Size = 100
Block 2: FREE | Size = 20
Block 3: P3 | Size = 300
Block 4: P7 | Size = 220
Block 5: FREE | Size = 30
Block 6: P5 | Size = 150

=== Best Fit ===
Execution Time: 2.00 us
Total Free Space : 50
Largest Free Block : 30
Free Holes : 2
External Fragmentation: 40.00%

Memory State:
Block 0: P1 | Size = 100
Block 1: P6 | Size = 100
Block 2: FREE | Size = 20
Block 3: P3 | Size = 300
Block 4: P7 | Size = 220
Block 5: FREE | Size = 30
Block 6: P5 | Size = 150

=== Worst Fit ===
Execution Time: 2.00 us
Total Free Space : 170
Largest Free Block : 100
Free Holes : 2
External Fragmentation: 41.18%

Memory State:
Block 0: P1 | Size = 100
Block 1: P6 | Size = 100
Block 2: FREE | Size = 100
Block 3: P3 | Size = 300
Block 4: P6 | Size = 100
Block 5: FREE | Size = 70
Block 6: P5 | Size = 150

=== Next Fit ===
Execution Time: 2.00 us
Total Free Space : 50
Largest Free Block : 30
Free Holes : 2
External Fragmentation: 40.00%

Memory State:
Block 0: P1 | Size = 100
Block 1: P6 | Size = 100
Block 2: FREE | Size = 20
Block 3: P3 | Size = 300
Block 4: P7 | Size = 220
Block 5: FREE | Size = 30
Block 6: P5 | Size = 150
asus@LAPTOP-VRS296DG:~/praktikum-memory/contiguous-allocation$ |
```

### 3.2.4 Penjelasan Implementasi

#### Proses Kompilasi Program

- Preprocessing
  - Header `stdio.h` dan `time.h` disertakan ke dalam kode sumber.
  - Makro `MAX_BLOCKS` didefinisikan dengan nilai 100.
  - Seluruh deklarasi struktur Block, variabel global (`memory`, `block_count`, `last_alloc_index`), dan prototype fungsi disiapkan sebelum kompilasi.
- Kompilasi
  - Kode C diterjemahkan ke bahasa assembly oleh compiler.
  - Struktur Block dan seluruh fungsi alokasi (`first_fit`, `best_fit`, `worst_fit`, `next_fit`) dianalisis.
  - Variabel global `memory[]`, `block_count`, dan `last_alloc_index` ditempatkan pada data segment.
  - Kode fungsi dialokasikan ke text segment.
- Assembly
  - Kode assembly diubah menjadi file objek yang berisi instruksi mesin.
  - Simbol fungsi dan variabel disimpan untuk tahap linking.
- Linking
  - File objek digabungkan dengan library standar C.
  - Fungsi pustaka seperti `printf()` dan `clock()` dihubungkan.
  - Berkas executable akhir dihasilkan.

## Proses Eksekusi Program

- Loading Program
  - Sistem operasi memuat program ke memori.
  - Text segment berisi kode fungsi benchmark dan algoritma alokasi.
  - Data segment berisi array memory[MAX\_BLOCKS] dan variabel kontrol.
  - Heap dan stack disiapkan untuk eksekusi.
- Eksekusi Awal
  - Fungsi main() dijalankan sebagai titik awal program.
  - Fungsi benchmark() dipanggil empat kali untuk setiap algoritma alokasi.
- Inisialisasi Memori
  - init\_memory(1000) membuat satu blok bebas berukuran 1000 byte.
  - Variabel block\_count dan last\_alloc\_index direset.
- Simulasi Alokasi
  - Proses dialokasikan dengan ukuran berbeda menggunakan algoritma yang diuji.
  - Jika blok lebih besar dari kebutuhan, blok dipecah (splitting).
  - block\_count bertambah dan array memory digeser.
- Dealokasi
  - Beberapa proses dibebaskan menggunakan deallocate().
  - Blok berubah status menjadi bebas (free).
- Alokasi Ulang
  - Proses baru dialokasikan ke ruang kosong yang tersedia.
  - Strategi pemilihan blok berbeda sesuai algoritma.
- Pengukuran Waktu
  - Waktu eksekusi dihitung menggunakan clock() dari time.h.
  - Hasil dikonversi ke mikrodetik.
- Analisis Fragmentasi
  - calculate\_fragmentation() menghitung total ruang bebas, lubang memori, dan fragmentasi eksternal.
- Visualisasi Memori
  - display\_memory() menampilkan status seluruh blok memori.
- Terminasi Program
  - Setelah seluruh benchmark selesai, program berhenti.

### 3.3 TUGAS 3

#### 3.3.1 Deskripsi Tugas

Tugas 3 berfokus pada pengembangan sistem paging lanjutan dengan mengimplementasikan algoritma page replacement FIFO dan LRU. Mahasiswa memodifikasi paging\_simulator.c agar mampu menangani kondisi ketika seluruh frame memori telah penuh. Sistem dilengkapi dengan perhitungan statistik seperti jumlah page fault, hit ratio, dan disk write untuk halaman yang dimodifikasi. Pengujian dilakukan menggunakan reference string tertentu dengan empat frame, kemudian hasil FIFO dan LRU dibandingkan. Selain itu, dilakukan pengujian Belady's Anomaly dengan variasi jumlah frame. Analisis difokuskan pada performa kedua algoritma serta kondisi kapan FIFO dapat lebih baik atau justru lebih buruk dibandingkan LRU.

#### 3.3.2 Source Code

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <math.h>
#include <time.h>
#include <limits.h>

#define FRAME_SIZE 256
#define MAX_FRAMES 16
#define PAGE_SIZE FRAME_SIZE

// Algoritma yang tersedia
typedef enum {
    FIFO,
    LRU
} Algorithm;

typedef struct {
    int page_number;
    int frame_number;
    int valid;
    int reference_bit;
    int modify_bit;
```

```

    int last_used; // Untuk LRU - timestamp penggunaan terakhir
} PageTableEntry;

typedef struct {
    int occupied;
    int page_number;
    int load_time; // Untuk FIFO - waktu pemuatan halaman
    int dirty; // Untuk statistik - halaman telah dimodifikasi
} Frame;

// Variabel global
Frame physical_memory[MAX_FRAMES];
PageTableEntry page_table[100];
int num_pages = 0;
int num_frames = 4; // Default 4 frames untuk test scenario
Algorithm current_algorithm = FIFO;

// Statistik
int page_faults = 0;
int page_hits = 0;
int disk_writes = 0;
int total_accesses = 0;

// Untuk FIFO
int fifo_queue[MAX_FRAMES];
int front = 0, rear = -1, queue_count = 0;

// Untuk LRU
int lru_counter = 0;

void initialize_system(int frames) {
    num_frames = frames;

    // Reset memory frames
    for (int i = 0; i < MAX_FRAMES; i++) {
        physical_memory[i].occupied = 0;
    }
}

```

```

    physical_memory[i].page_number = -1;
    physical_memory[i].load_time = 0;
    physical_memory[i].dirty = 0;
}

// Reset page table
for (int i = 0; i < 100; i++) {
    page_table[i].page_number = i;
    page_table[i].frame_number = -1;
    page_table[i].valid = 0;
    page_table[i].reference_bit = 0;
    page_table[i].modify_bit = 0;
    page_table[i].last_used = -1;
}

// Reset FIFO queue
front = 0;
rear = -1;
queue_count = 0;

// Reset LRU counter
lru_counter = 0;

// Reset statistik
page_faults = 0;
page_hits = 0;
disk_writes = 0;
total_accesses = 0;
}

void display_page_table() {
    printf("\n=== PAGE TABLE ===\n");
    printf("%-12s %-15s %-10s %-12s %-12s %-12s\n",
           "Page", "Frame", "Valid", "Referenced", "Modified", "Last Used");
    printf("-----\n");
}

```



```

for (int i = 0; i < num_pages; i++) {
    printf("%-12d %-15d %-10s %-12s %-12s %-12d\n",
        page_table[i].page_number,
        page_table[i].frame_number,
        page_table[i].valid ? "Yes" : "No",
        page_table[i].reference_bit ? "Yes" : "No",
        page_table[i].modify_bit ? "Yes" : "No",
        page_table[i].last_used);
}
printf("\n");
}

void display_physical_memory() {
    printf("=== PHYSICAL MEMORY (%d FRAMES) ===\n", num_frames);
    printf("%-10s %-10s %-15s %-10s %-10s\n",
        "Frame", "Status", "Page Number", "Load Time", "Dirty");
    printf("-----\n");

    for (int i = 0; i < num_frames; i++) {
        printf("%-10d %-10s", i, physical_memory[i].occupied ? "Occupied" : "Free");
        if (physical_memory[i].occupied) {
            printf("%-15d %-10d %-10s",
                physical_memory[i].page_number,
                physical_memory[i].load_time,
                physical_memory[i].dirty ? "Yes" : "No");
        }
        printf("\n");
    }
    printf("\n");
}

int find_free_frame() {
    for (int i = 0; i < num_frames; i++) {
        if (!physical_memory[i].occupied) {
            return i;
        }
    }
}

```

```

    }
    return -1;
}

void enqueue_fifo(int page_num) {
    rear = (rear + 1) % num_frames;
    fifo_queue[rear] = page_num;
    if (queue_count < num_frames) {
        queue_count++;
    }
}

int dequeue_fifo() {
    if (queue_count == 0) return -1;
    int page_num = fifo_queue[front];
    front = (front + 1) % num_frames;
    queue_count--;
    return page_num;
}

int find_lru_victim() {
    int victim_frame = -1;
    int min_last_used = INT_MAX;

    for (int i = 0; i < num_frames; i++) {
        if (physical_memory[i].occupied) {
            int page_num = physical_memory[i].page_number;
            if (page_table[page_num].last_used < min_last_used) {
                min_last_used = page_table[page_num].last_used;
                victim_frame = i;
            }
        }
    }

    return victim_frame;
}

```

```

int find_fifo_victim() {
    // FIFO menggunakan queue, jadi kita ambil dari depan
    if (queue_count == 0) return -1;

    int victim_page = fifo_queue[front];
    for (int i = 0; i < num_frames; i++) {
        if (physical_memory[i].occupied && physical_memory[i].page_number == victim_page) {
            return i;
        }
    }
    return -1;
}

int replace_page(int page_num, int write) {
    int victim_frame = -1;

    if (current_algorithm == FIFO) {
        victim_frame = find_fifo_victim();
    } else if (current_algorithm == LRU) {
        victim_frame = find_lru_victim();
    }

    if (victim_frame == -1) {
        return -1;
    }

    int victim_page = physical_memory[victim_frame].page_number;

    // Jika halaman korban telah dimodifikasi, perlu write ke disk
    if (page_table[victim_page].modify_bit && physical_memory[victim_frame].dirty) {
        disk_writes++;
        printf(" [DISK WRITE] Page %d (dirty) ditulis ke disk\n", victim_page);
    }

    // Update page table untuk halaman korban
    page_table[victim_page].valid = 0;
}

```

```

page_table[victim_page].frame_number = -1;
page_table[victim_page].modify_bit = 0;

// Update page table untuk halaman baru
page_table[page_num].frame_number = victim_frame;
page_table[page_num].valid = 1;
page_table[page_num].reference_bit = 1;
page_table[page_num].last_used = lru_counter++;
if (write) {
    page_table[page_num].modify_bit = 1;
}

// Update physical memory
physical_memory[victim_frame].page_number = page_num;
physical_memory[victim_frame].load_time = total_accesses;
physical_memory[victim_frame].dirty = write ? 1 : 0;

// Update FIFO queue jika menggunakan FIFO
if (current_algorithm == FIFO) {
    dequeue_fifo();
    enqueue_fifo(page_num);
}

return victim_frame;
}

int allocate_page(int page_num, int write) {
    total_accesses++;

    if (page_num >= 100) {
        printf("Error: Page number terlalu besar\n");
        return -1;
    }

    // Jika halaman sudah ada di memori (HIT)
    if (page_table[page_num].valid) {

```

```

    page_hits++;
    page_table[page_num].reference_bit = 1;
    page_table[page_num].last_used = lru_counter++;

    if (write) {
        page_table[page_num].modify_bit = 1;
        // Update dirty bit di physical memory
        int frame_num = page_table[page_num].frame_number;
        if (frame_num >= 0 && frame_num < num_frames) {
            physical_memory[frame_num].dirty = 1;
        }
    }

    return page_table[page_num].frame_number;
}

// PAGE FAULT
page_faults++;

int frame = find_free_frame();

if (frame == -1) {
    // Tidak ada frame kosong, perlu page replacement
    printf(" [PAGE REPLACEMENT] Menggunakan algoritma %s\n",
        current_algorithm == FIFO ? "FIFO" : "LRU");
    frame = replace_page(page_num, write);
    if (frame == -1) {
        printf("Error: Gagal melakukan page replacement\n");
        return -1;
    }
    printf(" Page %d menggantikan page di frame %d\n", page_num, frame);
} else {
    // Ada frame kosong
    page_table[page_num].frame_number = frame;
    page_table[page_num].valid = 1;
    page_table[page_num].reference_bit = 1;

```

```

    page_table[page_num].last_used = lru_counter++;
    if (write) {
        page_table[page_num].modify_bit = 1;
    }

    physical_memory[frame].occupied = 1;
    physical_memory[frame].page_number = page_num;
    physical_memory[frame].load_time = total_accesses;
    physical_memory[frame].dirty = write ? 1 : 0;

    // Untuk FIFO, tambahkan ke queue
    if (current_algorithm == FIFO) {
        enqueue_fifo(page_num);
    }

    printf("Page %d dialokasikan ke frame %d\n", page_num, frame);
}

if (page_num >= num_pages) {
    num_pages = page_num + 1;
}

return frame;
}

void display_statistics() {
    printf("\n==== STATISTIK ==== \n");
    printf("Algoritma: %s\n", current_algorithm == FIFO ? "FIFO" : "LRU");
    printf("Jumlah Frames: %d\n", num_frames);
    printf("Total Akses: %d\n", total_accesses);
    printf("Page Faults: %d\n", page_faults);
    printf("Page Hits: %d\n", page_hits);

    if (total_accesses > 0) {
        float fault_rate = (float)page_faults / total_accesses * 100;
        float hit_rate = (float)page_hits / total_accesses * 100;
    }
}

```

```

printf("Page Fault Rate: %.2f%%\n", fault_rate);
printf("Hit Rate: %.2f%%\n", hit_rate);
}

printf("Disk Writes: %d\n", disk_writes);
printf("\n");
}

void run_test_scenario(int frames, Algorithm algo) {

printf("\n=====
===== \n");
printf("|| TEST SCENARIO - %s dengan %d frames || \n",
      algo == FIFO ? "FIFO" : "LRU", frames);

printf("=====
===== \n\n");

initialize_system(frames);
current_algorithm = algo;

// Reference string dari soal
int reference_string[] = {7,0,1,2,0,3,0,4,2,3,0,3,2,1,2,0,1,7,0,1};
int length = sizeof(reference_string) / sizeof(reference_string[0]);

// Untuk menampilkan trace table
printf("Step\tReference\tFrame 0\tFrame 1\tFrame 2\tFrame 3\tFault?\n");
printf("-----\n");

for (int i = 0; i < length; i++) {
    int page_num = reference_string[i];
    int fault_before = page_faults;

    // Simulasi akses read
    allocate_page(page_num, 0);

```

```
// Tampilkan trace
printf("%d\t%d\t\t", i+1, page_num);

// Tampilkan isi frames
for (int f = 0; f < frames; f++) {
    if (physical_memory[f].occupied) {
        printf("%d\t", physical_memory[f].page_number);
    } else {
        printf("-\t");
    }
}

// Tampilkan apakah terjadi fault
if (page_faults > fault_before) {
    printf("FAULT");
} else {
    printf("HIT");
}
printf("\n");
}

display_statistics();
}

void compare_algorithms() {

printf("\n ════════════════════════════════════════════════════════════════  

═══════════ \n");
printf(" || PERBANDINGAN ALGORITMA (4 frames) || \n");

printf(" ════════════════════════════════════════════════════════════════  

═══════════ \n\n");

printf("%-15s %-20s %-15s %-20s\n",
"Algoritma", "Jumlah Page Fault", "Hit Ratio", "Disk Writes");
printf("-----\n");
```



```
// Jalankan FIFO
run_test_scenario(4, FIFO);
int fifo_faults = page_faults;
int fifo_writes = disk_writes;
float fifo_hit_rate = (float)page_hits / total_accesses * 100;

// Jalankan LRU
run_test_scenario(4, LRU);
int lru_faults = page_faults;
int lru_writes = disk_writes;
float lru_hit_rate = (float)page_hits / total_accesses * 100;

printf("\n=== TABEL PERBANDINGAN ===\n");
printf("%-15s %-20d %-15.2f%% %-20d\n",
        "FIFO", fifo_faults, fifo_hit_rate, fifo_writes);
printf("%-15s %-20d %-15.2f%% %-20d\n",
        "LRU", lru_faults, lru_hit_rate, lru_writes);
}

void belady_anomaly_test() {
printf("\n=====
===== \n");
printf("|| BELADY'S ANOMALY TEST || \n");

printf("=====
===== \n\n");

printf("Menguji apakah page fault berkurang seiring bertambahnya frames:\n\n");

// Test untuk 3, 4, dan 5 frames
int frames_list[] = {3, 4, 5};
int num_tests = sizeof(frames_list) / sizeof(frames_list[0]);

printf("%-10s %-15s %-15s\n", "Frames", "FIFO Faults", "LRU Faults");
```

```
printf("-----\n");

for (int i = 0; i < num_tests; i++) {
    int frames = frames_list[i];

    // Test FIFO
    run_test_scenario(frames, FIFO);
    int fifo_faults = page_faults;

    // Test LRU
    run_test_scenario(frames, LRU);
    int lru_faults = page_faults;

    printf("%-10d %-15d %-15d\n", frames, fifo_faults, lru_faults);
}

printf("\nANALISIS:\n");
printf("Belady's Anomaly terjadi ketika jumlah page fault BERTAMBAH\n");
printf("meskipun jumlah frame ditambah. Fenomena ini bisa terjadi\n");
printf("pada algoritma FIFO tetapi tidak pada LRU.\n");
}

void demo_scenario() {

printf("\n ════════════════════════════════════════════════════════════════════════════════  

═══════════ \n");
printf(" ║ DEMO SCENARIO: PAGING SYSTEM ║ \n");

printf(" ════════════════════════════════════════════════════════════════════════════════  

═══════════ \n\n");

initialize_system(4);
current_algorithm = FIFO;

printf("Skenario: Program berukuran 2048 bytes\n");
printf("Jumlah pages: %d pages\n\n", 2048 / PAGE_SIZE);
```

```

// Simulasi beberapa akses
int accesses[][2] = {
    {0, 0}, // Read address 0
    {300, 1}, // Write address 300
    {512, 0}, // Read address 512
    {100, 0}, // Read address 100
    {1024, 1} // Write address 1024
};

for (int i = 0; i < 5; i++) {
    int addr = accesses[i][0];
    int write = accesses[i][1];

    printf("\nAkses %s pada logical address %d:\n",
        write ? "WRITE" : "READ", addr);

    int page_num = addr / PAGE_SIZE;
    int frame = allocate_page(page_num, write);

    if (frame != -1) {
        printf(" Page %d -> Frame %d\n", page_num, frame);
    }
}

display_page_table();
display_physical_memory();
display_statistics();
}

int main() {
    int choice, address, write, frames, algo_choice;

    // Default initialization
    initialize_system(4);

```

```

while(1) {
    printf("\n=== ADVANCED PAGING SIMULATOR ===\n");
    printf("1. Set Jumlah Frames\n");
    printf("2. Set Algoritma (FIFO/LRU)\n");
    printf("3. Allocate Page\n");
    printf("4. Access Memory (Read)\n");
    printf("5. Access Memory (Write)\n");
    printf("6. Display Page Table\n");
    printf("7. Display Physical Memory\n");
    printf("8. Display Statistics\n");
    printf("9. Run Demo Scenario\n");
    printf("10. Run Test Scenario (Reference String)\n");
    printf("11. Compare FIFO vs LRU\n");
    printf("12. Belady's Anomaly Test\n");
    printf("13. Reset System\n");
    printf("14. Exit\n");
    printf("Pilihan: ");
    scanf("%d", &choice);

    switch(choice) {
        case 1:
            printf("Masukkan jumlah frames (1-%d): ", MAX_FRAMES);
            scanf("%d", &frames);
            if (frames > 0 && frames <= MAX_FRAMES) {
                initialize_system(frames);
                printf("Sistem direset dengan %d frames\n", frames);
            } else {
                printf("Jumlah frames tidak valid!\n");
            }
            break;

        case 2:
            printf("Pilih algoritma:\n");
            printf("1. FIFO\n");
            printf("2. LRU\n");
            printf("Pilihan: ");

```

```
scanf("%d", &algo_choice);
if (algo_choice == 1) {
    current_algorithm = FIFO;
    printf("Algoritma diubah menjadi FIFO\n");
} else if (algo_choice == 2) {
    current_algorithm = LRU;
    printf("Algoritma diubah menjadi LRU\n");
} else {
    printf("Pilihan tidak valid!\n");
}
break;
```

case 3:

```
printf("Masukkan page number: ");
scanf("%d", &address);
printf("Tipe akses (0=Read, 1=Write): ");
scanf("%d", &write);
allocate_page(address, write);
break;
```

case 4:

```
printf("Masukkan logical address: ");
scanf("%d", &address);
printf("Page %d diakses (READ)\n", address / PAGE_SIZE);
allocate_page(address / PAGE_SIZE, 0);
break;
```

case 5:

```
printf("Masukkan logical address: ");
scanf("%d", &address);
printf("Page %d diakses (WRITE)\n", address / PAGE_SIZE);
allocate_page(address / PAGE_SIZE, 1);
break;
```

case 6:

```
display_page_table();
```

```

        break;

    case 7:
        display_physical_memory();
        break;

    case 8:
        display_statistics();
        break;

    case 9:
        demo_scenario();
        break;

    case 10:
        printf("Masukkan jumlah frames untuk test: ");
        scanf("%d", &frames);
        printf("Pilih algoritma (1=FIFO, 2=LRU): ");
        scanf("%d", &algo_choice);
        if (frames > 0 && frames <= MAX_FRAMES && (algo_choice == 1 || algo_choice ==
2)) {
            run_test_scenario(frames, algo_choice == 1 ? FIFO : LRU);
        } else {
            printf("Input tidak valid!\n");
        }
        break;

    case 11:
        compare_algorithms();
        break;

    case 12:
        belady_anomaly_test();
        break;

    case 13:

```

```

        initialize_system(num_frames);

        printf("Sistem direset\n");

        break;

    case 14:

        printf("Keluar dari program...\n");

        return 0;

    default:

        printf("Pilihan tidak valid!\n");

    }

}

return 0;

}

```

### 3.3.3 Screenshot Output

```
asus@LAPTOP-VRS296DC:~/praktikum-memory/paging$ ./paging_simulator
```

```

=== ADVANCED PAGING SIMULATOR ===
1. Set Jumlah Frames
2. Set Algoritma (FIFO/LRU)
3. Allocate Page
4. Access Memory (Read)
5. Access Memory (Write)
6. Display Page Table
7. Display Physical Memory
8. Display Statistics
9. Run Demo Scenario
10. Run Test Scenario (Reference String)
11. Compare FIFO vs LRU
12. Belady's Anomaly Test
13. Reset System
14. Exit
Pilihan: 10
Masukkan jumlah frames untuk test: 4
Pilih algoritma (1=FIFO, 2=LRU): 1

```

```
TEST SCENARIO - FIFO dengan 4 frames
```

| Step                                          | Reference | Frame 0 | Frame 1 | Frame 2 | Frame 3 | Fault? |
|-----------------------------------------------|-----------|---------|---------|---------|---------|--------|
| Page 7 dialokasikan ke frame 0                |           |         |         |         |         |        |
| 1                                             | 7         | 7       | -       | -       | -       | FAULT  |
| Page 0 dialokasikan ke frame 1                |           |         |         |         |         |        |
| 2                                             | 0         | 7       | 0       | -       | -       | FAULT  |
| Page 1 dialokasikan ke frame 2                |           |         |         |         |         |        |
| 3                                             | 1         | 7       | 0       | 1       | -       | FAULT  |
| Page 2 dialokasikan ke frame 3                |           |         |         |         |         |        |
| 4                                             | 2         | 7       | 0       | 1       | 2       | FAULT  |
| 5                                             | 0         | 7       | 0       | 1       | 2       | HIT    |
| [PAGE REPLACEMENT] Menggunakan algoritma FIFO |           |         |         |         |         |        |
| Page 3 menggantikan page di frame 0           |           |         |         |         |         |        |
| 6                                             | 3         | 3       | 0       | 1       | 2       | FAULT  |
| 7                                             | 0         | 3       | 0       | 1       | 2       | HIT    |
| [PAGE REPLACEMENT] Menggunakan algoritma FIFO |           |         |         |         |         |        |
| Page 4 menggantikan page di frame 1           |           |         |         |         |         |        |
| 8                                             | 4         | 3       | 4       | 1       | 2       | FAULT  |
| 9                                             | 2         | 3       | 4       | 1       | 2       | HIT    |
| 10                                            | 3         | 3       | 4       | 1       | 2       | HIT    |
| [PAGE REPLACEMENT] Menggunakan algoritma FIFO |           |         |         |         |         |        |
| Page 0 menggantikan page di frame 2           |           |         |         |         |         |        |
| 11                                            | 0         | 3       | 4       | 0       | 2       | FAULT  |
| 12                                            | 3         | 3       | 4       | 0       | 2       | HIT    |
| 13                                            | 2         | 3       | 4       | 0       | 2       | HIT    |
| [PAGE REPLACEMENT] Menggunakan algoritma FIFO |           |         |         |         |         |        |
| Page 1 menggantikan page di frame 3           |           |         |         |         |         |        |
| 14                                            | 1         | 3       | 4       | 0       | 1       | FAULT  |
| [PAGE REPLACEMENT] Menggunakan algoritma FIFO |           |         |         |         |         |        |
| Page 2 menggantikan page di frame 0           |           |         |         |         |         |        |
| 15                                            | 2         | 2       | 4       | 0       | 1       | FAULT  |
| 16                                            | 0         | 2       | 4       | 0       | 1       | HIT    |
| 17                                            | 1         | 2       | 4       | 0       | 1       | HIT    |
| [PAGE REPLACEMENT] Menggunakan algoritma FIFO |           |         |         |         |         |        |
| Page 7 menggantikan page di frame 1           |           |         |         |         |         |        |
| 18                                            | 7         | 2       | 7       | 0       | 1       | FAULT  |
| 19                                            | 0         | 2       | 7       | 0       | 1       | HIT    |
| 20                                            | 1         | 2       | 7       | 0       | 1       | HIT    |

```

=== STATISTIK ===
Algoritma: FIFO

```

```

== ADVANCED PAGING SIMULATOR ==
1. Set Jumlah Frames
2. Set Algoritma (FIFO/LRU)
3. Allocate Page
4. Access Memory (Read)
5. Access Memory (Write)
6. Display Page Table
7. Display Physical Memory
8. Display Statistics
9. Run Demo Scenario
10. Run Test Scenario (Reference String)
11. Compare FIFO vs LRU
12. Belady's Anomaly Test
13. Reset System
14. Exit
Pilihan: 10
Masukkan jumlah frames untuk test: 4
Pilih algoritma (1=FIFO, 2=LRU): 2

```

TEST SCENARIO – LRU dengan 4 frames

| Step   | Reference                                    | Frame 0 | Frame 1 | Frame 2 | Frame 3 | Fault? |       |       |
|--------|----------------------------------------------|---------|---------|---------|---------|--------|-------|-------|
| Page 7 | dialokasikan ke frame 0                      | 7       | -       | -       | -       | FAULT  |       |       |
| Page 0 | dialokasikan ke frame 1                      | 0       | 0       | -       | -       | FAULT  |       |       |
| Page 1 | dialokasikan ke frame 2                      | 1       | 0       | 1       | -       | FAULT  |       |       |
| Page 2 | dialokasikan ke frame 3                      | 2       | 0       | 1       | 2       | HIT    |       |       |
| 5      | [PAGE REPLACEMENT] Menggunakan algoritma LRU |         |         |         |         |        |       |       |
| Page 3 | menggantikan page di frame 0                 | 3       | 0       | 1       | 2       | FAULT  |       |       |
| 7      | [PAGE REPLACEMENT] Menggunakan algoritma LRU |         |         |         |         |        |       |       |
| Page 4 | menggantikan page di frame 2                 | 4       | 0       | 4       | 2       | HIT    |       |       |
| 8      | [PAGE REPLACEMENT] Menggunakan algoritma LRU |         |         |         |         |        |       |       |
| 9      | menggantikan page di frame 1                 | 9       | 0       | 4       | 2       | HIT    |       |       |
| 10     | menggantikan page di frame 3                 | 10      | 0       | 4       | 2       | HIT    |       |       |
| 11     | menggantikan page di frame 0                 | 11      | 0       | 4       | 2       | HIT    |       |       |
| 12     | menggantikan page di frame 2                 | 12      | 0       | 4       | 2       | HIT    |       |       |
| 13     | menggantikan page di frame 1                 | 13      | 0       | 4       | 2       | HIT    |       |       |
| 14     | [PAGE REPLACEMENT] Menggunakan algoritma LRU |         |         |         |         |        |       |       |
| Page 1 | menggantikan page di frame 2                 | 1       | 3       | 0       | 1       | 2      | FAULT |       |
| 15     | menggantikan page di frame 0                 | 15      | 2       | 3       | 0       | 1      | 2     | HIT   |
| 16     | menggantikan page di frame 1                 | 16      | 0       | 3       | 0       | 1      | 2     | HIT   |
| 17     | menggantikan page di frame 3                 | 17      | 1       | 3       | 0       | 1      | 2     | HIT   |
| 18     | [PAGE REPLACEMENT] Menggunakan algoritma LRU |         |         |         |         |        |       |       |
| Page 7 | menggantikan page di frame 0                 | 7       | 1       | 3       | 0       | 1      | 2     | FAULT |

| Page | 7 | menggantikan | page | di | frame | 0 |       |  |  |
|------|---|--------------|------|----|-------|---|-------|--|--|
| 18   | 7 |              | 7    | 0  | 1     | 2 | FAULT |  |  |
| 19   | 0 |              | 7    | 0  | 1     | 2 | HIT   |  |  |
| 20   | 1 |              | 7    | 0  | 1     | 2 | HIT   |  |  |

```

=== ADVANCED PAGING SIMULATOR ===
1. Set Jumlah Frames
2. Set Algoritma (FIFO/LRU)
3. Allocate Page
4. Access Memory (Read)
5. Access Memory (Write)
6. Display Page Table
7. Display Physical Memory
8. Display Statistics
9. Run Demo Scenario
10. Run Test Scenario (Reference String)
11. Compare FIFO vs LRU
12. Belady's Anomaly Test
13. Reset System
14. Exit
Pilihan: 11

```

PERBANDINGAN ALGORITMA (4 frames)

| Algoritma | Jumlah Page Fault | Hit Ratio | Disk Writes |
|-----------|-------------------|-----------|-------------|
|-----------|-------------------|-----------|-------------|

TEST SCENARIO – FIFO dengan 4 frames

| Step                                          | Reference                    | Frame 0 | Frame 1 | Frame 2 | Frame 3 | Fault? |
|-----------------------------------------------|------------------------------|---------|---------|---------|---------|--------|
| Page 7                                        | dialokasikan ke frame 0      |         |         |         |         |        |
| 1                                             | 7                            | 7       | -       | -       | -       | FAULT  |
| Page 0                                        | dialokasikan ke frame 1      |         |         |         |         |        |
| 2                                             | 0                            | 0       | -       | -       | -       | FAULT  |
| Page 1                                        | dialokasikan ke frame 2      |         |         |         |         |        |
| 3                                             | 1                            | 7       | 0       | 1       | -       | FAULT  |
| Page 2                                        | dialokasikan ke frame 3      |         |         |         |         |        |
| 4                                             | 2                            | 7       | 0       | 1       | 2       | FAULT  |
| 5                                             | 0                            | 7       | 0       | 1       | 2       | HIT    |
| [PAGE REPLACEMENT] Menggunakan algoritma FIFO |                              |         |         |         |         |        |
| Page 3                                        | menggantikan page di frame 0 |         |         |         |         |        |
| 6                                             | 3                            | 3       | 0       | 1       | 2       | FAULT  |
| 7                                             | 0                            | 3       | 0       | 1       | 2       | HIT    |
| [PAGE REPLACEMENT] Menggunakan algoritma FIFO |                              |         |         |         |         |        |
| Page 4                                        | menggantikan page di frame 1 |         |         |         |         |        |
| 8                                             | 4                            | 3       | 4       | 1       | 2       | FAULT  |
| 9                                             | 2                            | 3       | 4       | 1       | 2       | HIT    |
| 10                                            | 3                            | 3       | 4       | 1       | 2       | HIT    |
| [PAGE REPLACEMENT] Menggunakan algoritma FIFO |                              |         |         |         |         |        |
| Page 0                                        | menggantikan page di frame 2 |         |         |         |         |        |



```

7      0      3      0      1      2      HIT
[PAGE REPLACEMENT] Menggunakan algoritma FIFO
Page 4 menggantikan page di frame 1
8      4      3      4      1      2      FAULT
9      2      3      4      1      2      HIT
10     3      3      4      1      2      HIT
[PAGE REPLACEMENT] Menggunakan algoritma FIFO
Page 0 menggantikan page di frame 2
11     0      3      4      0      2      FAULT
12     3      3      4      0      2      HIT
13     2      3      4      0      2      HIT
[PAGE REPLACEMENT] Menggunakan algoritma FIFO
Page 1 menggantikan page di frame 3
14     1      3      4      0      1      FAULT
[PAGE REPLACEMENT] Menggunakan algoritma FIFO
Page 2 menggantikan page di frame 0
15     2      2      4      0      1      FAULT
16     0      2      4      0      1      HIT
17     1      2      4      0      1      HIT
[PAGE REPLACEMENT] Menggunakan algoritma FIFO
Page 7 menggantikan page di frame 1
18     7      2      7      0      1      FAULT
19     0      2      7      0      1      HIT
20     1      2      7      0      1      HIT

=== STATISTIK ===
Algoritma: FIFO
Jumlah Frames: 4
Total Akses: 20
Page Faults: 10
Page Hits: 10
Page Fault Rate: 50.00%
Hit Rate: 50.00%
Disk Writes: 0

```

#### TEST SCENARIO - LRU dengan 4 frames

| Step                                         | Reference | Frame 0 | Frame 1 | Frame 2 | Frame 3 | Fault? |
|----------------------------------------------|-----------|---------|---------|---------|---------|--------|
| Page 7 dialokasikan ke frame 0               |           |         |         |         |         |        |
| 1                                            | 7         | -       | -       | -       | -       | FAULT  |
| Page 0 dialokasikan ke frame 1               |           |         |         |         |         |        |
| 2                                            | 0         | 7       | 0       | -       | -       | FAULT  |
| Page 1 dialokasikan ke frame 2               |           |         |         |         |         |        |
| 3                                            | 1         | 7       | 0       | 1       | -       | FAULT  |
| Page 2 dialokasikan ke frame 3               |           |         |         |         |         |        |
| 4                                            | 2         | 7       | 0       | 1       | 2       | FAULT  |
| 5                                            | 0         | 7       | 0       | 1       | 2       | HIT    |
| [PAGE REPLACEMENT] Menggunakan algoritma LRU |           |         |         |         |         |        |
| Page 3 menggantikan page di frame 0          |           |         |         |         |         |        |
| 6                                            | 3         | 3       | 0       | 1       | 2       | FAULT  |
| 7                                            | 0         | 3       | 0       | 1       | 2       | HIT    |
| [PAGE REPLACEMENT] Menggunakan algoritma LRU |           |         |         |         |         |        |
| Page 4 menggantikan page di frame 2          |           |         |         |         |         |        |
| 8                                            | 4         | 3       | 0       | 4       | 2       | FAULT  |
| 9                                            | 2         | 3       | 0       | 4       | 2       | HIT    |
| 10                                           | 3         | 3       | 0       | 4       | 2       | HIT    |
| 11                                           | 0         | 3       | 0       | 4       | 2       | HIT    |
| 12                                           | 3         | 3       | 0       | 4       | 2       | HIT    |
| 13                                           | 2         | 3       | 0       | 4       | 2       | HIT    |
| [PAGE REPLACEMENT] Menggunakan algoritma LRU |           |         |         |         |         |        |
| Page 1 menggantikan page di frame 2          |           |         |         |         |         |        |
| 14                                           | 1         | 3       | 0       | 1       | 2       | FAULT  |

```
[PAGE REPLACEMENT] Menggunakan algoritma LRU
Page 1 menggantikan page di frame 2
14 1      3      0      1      2      FAULT
15 2      3      0      1      2      HIT
16 0      3      0      1      2      HIT
17 1      3      0      1      2      HIT
[PAGE REPLACEMENT] Menggunakan algoritma LRU
Page 7 menggantikan page di frame 0
18 7      7      0      1      2      FAULT
19 0      7      0      1      2      HIT
20 1      7      0      1      2      HIT
```

```
=== STATISTIK ===
Algoritma: LRU
Jumlah Frames: 4
Total Akses: 20
Page Faults: 8
Page Hits: 12
Page Fault Rate: 40.00%
Hit Rate: 60.00%
Disk Writes: 0
```

```
=== TABEL PERBANDINGAN ===
FIFO      10      50.00      % 0
LRU       8      60.00      % 0
```

```
=== ADVANCED PAGING SIMULATOR ===
1. Set Jumlah Frames
2. Set Algoritma (FIFO/LRU)
3. Allocate Page
4. Access Memory (Read)
5. Access Memory (Write)
6. Display Page Table
7. Display Physical Memory
8. Display Statistics
9. Run Demo Scenario
10. Run Test Scenario (Reference String)
11. Compare FIFO vs LRU
12. Belady's Anomaly Test
13. Reset System
14. Exit
Pilihan: 12
```

BELADY'S ANOMALY TEST

Menguji apakah page fault berkurang seiring bertambahnya frames:

Frames      FIFO Faults      LRU Faults

TEST SCENARIO - FIFO dengan 3 frames

```
Step  Reference      Frame 0 Frame 1 Frame 2 Frame 3 Fault?
Page 7 dialokasikan ke frame 0
1 7      -      -      -      -      FAULT
Page 0 dialokasikan ke frame 1
2 0      7      -      -      -      FAULT
Page 1 dialokasikan ke frame 2
3 1      7      0      -      -      FAULT
[PAGE REPLACEMENT] Menggunakan algoritma FIFO
```

TEST SCENARIO - FIFO dengan 3 frames

```
Step  Reference      Frame 0 Frame 1 Frame 2 Frame 3 Fault?
Page 7 dialokasikan ke frame 0
1 7      -      -      -      -      FAULT
Page 0 dialokasikan ke frame 1
2 0      7      -      -      -      FAULT
Page 1 dialokasikan ke frame 2
3 1      7      0      -      -      FAULT
[PAGE REPLACEMENT] Menggunakan algoritma FIFO
Page 2 menggantikan page di frame 0
4 2      2      0      -      -      FAULT
5 0      2      0      1      -      HIT
[PAGE REPLACEMENT] Menggunakan algoritma FIFO
Page 3 menggantikan page di frame 1
6 3      2      3      1      -      FAULT
[PAGE REPLACEMENT] Menggunakan algoritma FIFO
Page 0 menggantikan page di frame 2
7 0      2      3      0      -      FAULT
[PAGE REPLACEMENT] Menggunakan algoritma FIFO
Page 4 menggantikan page di frame 0
8 4      4      3      0      -      FAULT
[PAGE REPLACEMENT] Menggunakan algoritma FIFO
Page 2 menggantikan page di frame 1
9 2      4      2      0      -      FAULT
[PAGE REPLACEMENT] Menggunakan algoritma FIFO
Page 3 menggantikan page di frame 2
10 3      4      2      3      -      FAULT
[PAGE REPLACEMENT] Menggunakan algoritma FIFO
Page 0 menggantikan page di frame 0
11 0      0      2      3      -      FAULT
12 3      0      2      3      -      HIT
13 2      0      2      3      -      HIT
[PAGE REPLACEMENT] Menggunakan algoritma FIFO
Page 1 menggantikan page di frame 1
14 1      0      1      3      -      FAULT
[PAGE REPLACEMENT] Menggunakan algoritma FIFO
Page 2 menggantikan page di frame 2
15 2      0      1      2      -      FAULT
16 0      0      1      2      -      HIT
17 1      0      1      2      -      HIT
[PAGE REPLACEMENT] Menggunakan algoritma FIFO
Page 7 menggantikan page di frame 0
18 7      7      1      2      -      FAULT
[PAGE REPLACEMENT] Menggunakan algoritma FIFO
Page 0 menggantikan page di frame 1
19 0      7      0      2      -      FAULT
[PAGE REPLACEMENT] Menggunakan algoritma FIFO
Page 1 menggantikan page di frame 2
20 1      7      0      1      -      FAULT
```

```
=== STATISTIK ===
Algoritma: FIFO
Jumlah Frames: 3
Total Akses: 20
Page Faults: 15
Page Hits: 5
Page Fault Rate: 75.00%
Hit Rate: 25.00%
Disk Writes: 0
```

```
=== STATISTIK ===
Algoritma: FIFO
Jumlah Frames: 3
Total Akses: 20
Page Faults: 15
Page Hits: 5
Page Fault Rate: 75.00%
Hit Rate: 25.00%
Disk Writes: 0
```

#### TEST SCENARIO - LRU dengan 3 frames

| Step                                         | Reference | Frame 0 | Frame 1 | Frame 2 | Frame 3 | Fault? |
|----------------------------------------------|-----------|---------|---------|---------|---------|--------|
| Page 7 dialokasikan ke frame 0               |           |         |         |         |         |        |
| 1                                            | 7         | 7       | -       | -       | -       | FAULT  |
| Page 0 dialokasikan ke frame 1               |           |         |         |         |         |        |
| 2                                            | 0         | 7       | 0       | -       | -       | FAULT  |
| Page 1 dialokasikan ke frame 2               |           |         |         |         |         |        |
| 3                                            | 1         | 7       | 0       | 1       | -       | FAULT  |
| [PAGE REPLACEMENT] Menggunakan algoritma LRU |           |         |         |         |         |        |
| Page 2 menggantikan page di frame 0          |           |         |         |         |         |        |
| 4                                            | 2         | 2       | 0       | 1       | -       | FAULT  |
| 5                                            | 0         | 2       | 0       | 1       | -       | HIT    |
| [PAGE REPLACEMENT] Menggunakan algoritma LRU |           |         |         |         |         |        |
| Page 3 menggantikan page di frame 2          |           |         |         |         |         |        |
| 6                                            | 3         | 2       | 0       | 3       | -       | FAULT  |
| 7                                            | 0         | 2       | 0       | 3       | -       | HIT    |
| [PAGE REPLACEMENT] Menggunakan algoritma LRU |           |         |         |         |         |        |
| Page 4 menggantikan page di frame 0          |           |         |         |         |         |        |
| 8                                            | 4         | 4       | 0       | 3       | -       | FAULT  |
| [PAGE REPLACEMENT] Menggunakan algoritma LRU |           |         |         |         |         |        |
| Page 2 menggantikan page di frame 2          |           |         |         |         |         |        |
| 9                                            | 2         | 4       | 0       | 2       | -       | FAULT  |
| [PAGE REPLACEMENT] Menggunakan algoritma LRU |           |         |         |         |         |        |
| Page 3 menggantikan page di frame 1          |           |         |         |         |         |        |
| 10                                           | 3         | 4       | 3       | 2       | -       | FAULT  |
| [PAGE REPLACEMENT] Menggunakan algoritma LRU |           |         |         |         |         |        |
| Page 0 menggantikan page di frame 0          |           |         |         |         |         |        |
| 11                                           | 0         | 0       | 3       | 2       | -       | FAULT  |
| 12                                           | 3         | 0       | 3       | 2       | -       | HIT    |
| 13                                           | 2         | 0       | 3       | 2       | -       | HIT    |
| [PAGE REPLACEMENT] Menggunakan algoritma LRU |           |         |         |         |         |        |
| Page 1 menggantikan page di frame 0          |           |         |         |         |         |        |
| 14                                           | 1         | 1       | 3       | 2       | -       | FAULT  |
| 15                                           | 2         | 1       | 3       | 2       | -       | HIT    |
| [PAGE REPLACEMENT] Menggunakan algoritma LRU |           |         |         |         |         |        |
| Page 0 menggantikan page di frame 1          |           |         |         |         |         |        |
| 16                                           | 0         | 1       | 0       | 2       | -       | FAULT  |
| 17                                           | 1         | 1       | 0       | 2       | -       | HIT    |
| [PAGE REPLACEMENT] Menggunakan algoritma LRU |           |         |         |         |         |        |
| Page 7 menggantikan page di frame 2          |           |         |         |         |         |        |
| 18                                           | 7         | 1       | 0       | 7       | -       | FAULT  |
| 19                                           | 0         | 1       | 0       | 7       | -       | HIT    |
| 20                                           | 1         | 1       | 0       | 7       | -       | HIT    |

```
=== STATISTIK ===
Algoritma: LRU
Jumlah Frames: 3
Total Akses: 20
Page Faults: 12
Page Hits: 8
Page Fault Rate: 60.00%
```

#### TEST SCENARIO - FIFO dengan 4 frames

| Step                                          | Reference | Frame 0 | Frame 1 | Frame 2 | Frame 3 | Fault? |
|-----------------------------------------------|-----------|---------|---------|---------|---------|--------|
| Page 7 dialokasikan ke frame 0                |           |         |         |         |         |        |
| 1                                             | 7         | 7       | -       | -       | -       | FAULT  |
| Page 0 dialokasikan ke frame 1                |           |         |         |         |         |        |
| 2                                             | 0         | 7       | 0       | -       | -       | FAULT  |
| Page 1 dialokasikan ke frame 2                |           |         |         |         |         |        |
| 3                                             | 1         | 7       | 0       | 1       | -       | FAULT  |
| Page 2 dialokasikan ke frame 3                |           |         |         |         |         |        |
| 4                                             | 2         | 7       | 0       | 1       | 2       | FAULT  |
| 5                                             | 0         | 7       | 0       | 1       | 2       | HIT    |
| [PAGE REPLACEMENT] Menggunakan algoritma FIFO |           |         |         |         |         |        |
| Page 3 menggantikan page di frame 0           |           |         |         |         |         |        |
| 6                                             | 3         | 3       | 0       | 1       | 2       | FAULT  |
| 7                                             | 0         | 3       | 0       | 1       | 2       | HIT    |
| [PAGE REPLACEMENT] Menggunakan algoritma FIFO |           |         |         |         |         |        |
| Page 4 menggantikan page di frame 1           |           |         |         |         |         |        |
| 8                                             | 4         | 3       | 4       | 1       | 2       | FAULT  |
| 9                                             | 2         | 3       | 4       | 1       | 2       | HIT    |
| 10                                            | 3         | 3       | 4       | 1       | 2       | HIT    |
| [PAGE REPLACEMENT] Menggunakan algoritma FIFO |           |         |         |         |         |        |
| Page 0 menggantikan page di frame 2           |           |         |         |         |         |        |
| 11                                            | 0         | 3       | 4       | 0       | 2       | FAULT  |
| 12                                            | 3         | 3       | 4       | 0       | 2       | HIT    |
| 13                                            | 2         | 3       | 4       | 0       | 2       | HIT    |
| [PAGE REPLACEMENT] Menggunakan algoritma FIFO |           |         |         |         |         |        |
| Page 1 menggantikan page di frame 3           |           |         |         |         |         |        |
| 14                                            | 1         | 3       | 4       | 0       | 1       | FAULT  |
| [PAGE REPLACEMENT] Menggunakan algoritma FIFO |           |         |         |         |         |        |
| Page 2 menggantikan page di frame 0           |           |         |         |         |         |        |
| 15                                            | 2         | 2       | 4       | 0       | 1       | FAULT  |
| 16                                            | 0         | 2       | 4       | 0       | 1       | HIT    |
| 17                                            | 1         | 2       | 4       | 0       | 1       | HIT    |
| [PAGE REPLACEMENT] Menggunakan algoritma FIFO |           |         |         |         |         |        |
| Page 7 menggantikan page di frame 1           |           |         |         |         |         |        |
| 18                                            | 7         | 2       | 7       | 0       | 1       | FAULT  |
| 19                                            | 0         | 2       | 7       | 0       | 1       | HIT    |
| 20                                            | 1         | 2       | 7       | 0       | 1       | HIT    |

```
=== STATISTIK ===
Algoritma: FIFO
Jumlah Frames: 4
Total Akses: 20
Page Faults: 10
Page Hits: 10
Page Fault Rate: 50.00%
Hit Rate: 50.00%
Disk Writes: 0
```

#### TEST SCENARIO - LRU dengan 4 frames

| Step                           | Reference | Frame 0 | Frame 1 | Frame 2 | Frame 3 | Fault? |
|--------------------------------|-----------|---------|---------|---------|---------|--------|
| Page 7 dialokasikan ke frame 0 |           |         |         |         |         |        |
| 1                              | 7         | 7       | -       | -       | -       | FAULT  |
| Page 0 dialokasikan ke frame 1 |           |         |         |         |         |        |
| 2                              | 0         | 7       | 0       | -       | -       | FAULT  |
| Page 1 dialokasikan ke frame 2 |           |         |         |         |         |        |
| 3                              | 1         | 7       | 0       | 1       | -       | FAULT  |

```
TEST SCENARIO - FIFO dengan 5 frames

Step  Reference      Frame 0 Frame 1 Frame 2 Frame 3 Fault?
Page 7 dialokasikan ke frame 0
1      7      7      -      -      -      -      FAULT
Page 0 dialokasikan ke frame 1
2      0      7      0      -      -      -      FAULT
Page 1 dialokasikan ke frame 2
3      1      7      0      1      -      -      FAULT
Page 2 dialokasikan ke frame 3
4      2      7      0      1      2      -      FAULT
5      0      7      0      1      2      -      HIT
Page 3 dialokasikan ke frame 4
6      3      7      0      1      2      3      FAULT
7      0      7      0      1      2      3      HIT
[PAGE REPLACEMENT] Menggunakan algoritma FIFO
Page 4 menggantikan page di frame 0
8      4      4      0      1      2      3      FAULT
9      2      4      0      1      2      3      HIT
10     3      4      0      1      2      3      HIT
11     0      4      0      1      2      3      HIT
12     3      4      0      1      2      3      HIT
13     2      4      0      1      2      3      HIT
14     1      4      0      1      2      3      HIT
15     2      4      0      1      2      3      HIT
16     0      4      0      1      2      3      HIT
17     1      4      0      1      2      3      HIT
[PAGE REPLACEMENT] Menggunakan algoritma FIFO
Page 7 menggantikan page di frame 1
18     7      4      7      1      2      3      FAULT
[PAGE REPLACEMENT] Menggunakan algoritma FIFO
Page 0 menggantikan page di frame 2
19     0      4      7      0      2      3      FAULT
[PAGE REPLACEMENT] Menggunakan algoritma FIFO
Page 1 menggantikan page di frame 3
20     1      4      7      0      1      3      FAULT

=== STATISTIK ===
Algoritma: FIFO
Jumlah Frames: 5
Total Akses: 20
Page Faults: 9
Page Hits: 11
Page Fault Rate: 45.00%
Hit Rate: 55.00%
Disk Writes: 0

TEST SCENARIO - LRU dengan 5 frames

Step  Reference      Frame 0 Frame 1 Frame 2 Frame 3 Fault?
Page 7 dialokasikan ke frame 0
1      7      7      -      -      -      -      FAULT
Page 0 dialokasikan ke frame 1
2      0      7      0      -      -      -      FAULT
Page 1 dialokasikan ke frame 2
3      1      7      0      1      -      -      FAULT
Page 2 dialokasikan ke frame 3
4      2      7      0      1      2      -      FAULT

Page 0 dialokasikan ke frame 1
2      0      7      0      -      -      -      FAULT
Page 1 dialokasikan ke frame 2
3      1      7      0      1      -      -      FAULT
Page 2 dialokasikan ke frame 3
4      2      7      0      1      2      -      FAULT
5      0      7      0      1      2      -      HIT
Page 3 dialokasikan ke frame 4
6      3      7      0      1      2      3      FAULT
7      0      7      0      1      2      3      HIT
[PAGE REPLACEMENT] Menggunakan algoritma LRU
Page 4 menggantikan page di frame 0
8      4      4      0      1      2      3      FAULT
9      2      4      0      1      2      3      HIT
10     3      4      0      1      2      3      HIT
11     0      4      0      1      2      3      HIT
12     3      4      0      1      2      3      HIT
13     2      4      0      1      2      3      HIT
14     1      4      0      1      2      3      HIT
15     2      4      0      1      2      3      HIT
16     0      4      0      1      2      3      HIT
17     1      4      0      1      2      3      HIT
[PAGE REPLACEMENT] Menggunakan algoritma LRU
Page 7 menggantikan page di frame 0
18     7      7      0      1      2      3      FAULT
19     0      7      0      1      2      3      HIT
20     1      7      0      1      2      3      HIT

=== STATISTIK ===
Algoritma: LRU
Jumlah Frames: 5
Total Akses: 20
Page Faults: 7
Page Hits: 13
Page Fault Rate: 35.00%
Hit Rate: 65.00%
Disk Writes: 0

5      9      7

ANALISIS:
Belady's Anomaly terjadi ketika jumlah page fault BERTAMBAH
meskipun jumlah frame ditambah. Fenomena ini bisa terjadi
pada algoritma FIFO tetapi tidak pada LRU.

=== ADVANCED PAGING SIMULATOR ===
1. Set Jumlah Frames
2. Set Algoritma (FIFO/LRU)
3. Allocate Page
4. Access Memory (Read)
5. Access Memory (Write)
6. Display Page Table
7. Display Physical Memory
8. Display Statistics
9. Run Demo Scenario
10. Run Test Scenario (Reference String)
11. Compare FIFO vs LRU
12. Belady's Anomaly Test
13. Reset System
14. Exit
Pilihan: 14
Keluar dari program...
asus@LAPTOP-VRS286DG:~/praktikum-memory/paging$
```

### 3.3.4 Penjelasan Implementasi

#### Proses Kompilasi Program

- Preprocessing
  - Header `stdio.h`, `stdlib.h`, `string.h`, `math.h`, `time.h`, dan `limits.h` dimasukkan ke dalam kode sumber.
  - Makro `FRAME_SIZE`, `PAGE_SIZE`, dan `MAX_FRAMES` didefinisikan.
  - Definisi enum `Algorithm` (`FIFO` dan `LRU`) serta struktur `PageTableEntry` dan `Frame` dipersiapkan.
  - Deklarasi variabel global seperti `physical_memory`, `page_table`, dan statistik paging diproses.
- Kompilasi
  - Kode C diterjemahkan ke bahasa assembly.
  - Seluruh fungsi paging (`allocate_page`, `replace_page`, `find_fifo_victim`, `find_lru_victim`, dll.) dianalisis.
  - Variabel global dialokasikan ke data segment.
  - Kode fungsi dialokasikan ke text segment.
- Assembly
  - Kode assembly dikonversi menjadi file objek berisi instruksi mesin.
  - Informasi simbol untuk fungsi dan variabel disiapkan untuk proses linking.
- Linking
  - File objek digabungkan dengan library standar C.
  - Fungsi pustaka seperti `printf()`, `scanf()`, dan `clock()` dihubungkan.
  - File executable akhir dihasilkan dan siap dijalankan.

#### Proses Eksekusi Program

- Loading Program
  - Sistem operasi memuat executable ke memori.
  - Text segment berisi kode simulator paging.
  - Data segment menyimpan page table, physical memory, dan statistik.
  - Stack disiapkan untuk pemanggilan fungsi, heap untuk alokasi internal.
- Inisialisasi Awal
  - Fungsi `main()` dijalankan.
  - `initialize_system(4)` dipanggil untuk mengatur sistem dengan 4 frame awal.
  - Semua frame dikosongkan dan page table di-reset.
- Menu Interaktif
  - Program menampilkan menu simulator paging.
  - Pengguna dapat memilih jumlah frame, algoritma FIFO/LRU, serta jenis akses memori.

- Akses Memori
  - Logical address diterjemahkan menjadi page number ( $\text{address} / \text{PAGE\_SIZE}$ ).
  - Fungsi `allocate_page()` dipanggil untuk setiap akses read/write.
  - Jika halaman valid → page hit.
  - Jika tidak valid → page fault.
- Page Fault Handling
  - Sistem mencari frame kosong dengan `find_free_frame()`.
  - Jika frame penuh, dilakukan page replacement:
    - FIFO: menggunakan antrian halaman tertua.
    - LRU: memilih halaman dengan `last_used` terkecil.
  - Jika halaman korban dirty, dilakukan simulasi disk write.
- Update Struktur Data
  - Page table diperbarui (valid bit, frame number, reference bit, modify bit).
  - Physical memory diperbarui sesuai halaman yang dimuat.
  - Statistik (`page_faults`, `page_hits`, `disk_writes`) ditambah.
- Simulasi dan Analisis
  - `run_test_scenario()` menjalankan reference string dan menampilkan trace frame.
  - `compare_algorithms()` membandingkan FIFO dan LRU.
  - `belady_anomaly_test()` menguji anomali Belady pada FIFO.
  - `demo_scenario()` menunjukkan translasi alamat logis ke fisik.
- Visualisasi
  - `display_page_table()` menampilkan isi tabel halaman.
  - `display_physical_memory()` menampilkan kondisi frame.
  - `display_statistics()` menampilkan hit rate, fault rate, dan disk write.
- Terminasi Program
  - Program berhenti saat pengguna memilih menu Exit.
  - Sistem operasi membersihkan resource dan program keluar dengan status sukses (`return 0`)

## 3.4 TUGAS 4

### 3.4.1 Deskripsi Tugas

Tugas 4 bertujuan mengimplementasikan dan membandingkan berbagai struktur page table, yaitu single-level, two-level, three-level, dan inverted page table. Mahasiswa mengembangkan sistem three-level page table untuk memahami pembagian alamat virtual secara hierarkis serta mengimplementasikan inverted page table yang memetakan frame ke proses dan halaman. Analisis dilakukan dengan membandingkan memory overhead, jumlah entri, ukuran tabel, dan efisiensi ruang masing-masing struktur. Selain itu, kecepatan translasi alamat dianalisis berdasarkan jumlah akses memori dan pengaruh TLB. Tugas ini juga membahas kecocokan penggunaan struktur page table pada sistem 32-bit dan 64-bit serta kondisi optimal penggunaan inverted page table.

### 3.4.2 Source Code

```
#include <stdio.h>
#include <stdlib.h>

#define L1_BITS 16
#define L2_BITS 8
#define L3_BITS 8
#define OFFSET_BITS 12

#define NUM_FRAMES 1024 // untuk inverted page table

/* ===== */
/* THREE-LEVEL PAGE TABLE */
/* ===== */

typedef struct {
    int frame_number;
    int valid;
    int read, write, execute;
} L3_Entry;

typedef struct {
    L3_Entry *entries;
```

```

    int valid;
} L2_Entry;

typedef struct {
    L2_Entry *entries;
    int valid;
} L1_Entry;

L1_Entry *L1;

/* Ambil index dari virtual address */
unsigned int get_l1(unsigned long va) {
    return (va >> (L2_BITS + L3_BITS + OFFSET_BITS)) & ((1 << L1_BITS) - 1);
}

unsigned int get_l2(unsigned long va) {
    return (va >> (L3_BITS + OFFSET_BITS)) & ((1 << L2_BITS) - 1);
}

unsigned int get_l3(unsigned long va) {
    return (va >> OFFSET_BITS) & ((1 << L3_BITS) - 1);
}

/* Alokasi page */
void allocate_3level(unsigned long va, int frame) {
    int l1 = get_l1(va);
    int l2 = get_l2(va);
    int l3 = get_l3(va);

    if (!L1[l1].valid) {
        L1[l1].entries = calloc(1 << L2_BITS, sizeof(L2_Entry));
        L1[l1].valid = 1;
    }

    if (!L1[l1].entries[l2].valid) {
        L1[l1].entries[l2].entries = calloc(1 << L3_BITS, sizeof(L3_Entry));
    }
}

```



```

        L1[l1].entries[l2].valid = 1;
    }

    L1[l1].entries[l2].entries[l3].frame_number = frame;
    L1[l1].entries[l2].entries[l3].valid = 1;
}

/* Translasi alamat */
int translate_3level(unsigned long va) {
    int l1 = get_l1(va);
    int l2 = get_l2(va);
    int l3 = get_l3(va);
    int offset = va & ((1 << OFFSET_BITS) - 1);

    if (!L1[l1].valid) return -1;
    if (!L1[l1].entries[l2].valid) return -1;
    if (!L1[l1].entries[l2].entries[l3].valid) return -1;

    return (L1[l1].entries[l2].entries[l3].frame_number << OFFSET_BITS) | offset;
}

/* Hitung memory overhead */
void calculate_memory_overhead() {
    int l1_count = 1 << L1_BITS;
    int l2_count = 0, l3_count = 0;

    for (int i = 0; i < l1_count; i++) {
        if (L1[i].valid) {
            l2_count++;
            for (int j = 0; j < (1 << L2_BITS); j++) {
                if (L1[i].entries[j].valid) {
                    l3_count++;
                }
            }
        }
    }
}

```

```
printf("\n=== MEMORY OVERHEAD THREE-LEVEL ===\n");
printf("L1 tables : %d\n", l1_count);
printf("L2 tables aktif : %d\n", l2_count);
printf("L3 tables aktif : %d\n", l3_count);
}
```

```
/* ===== */
/* INVERTED PAGE TABLE      */
/* ===== */
```

```
typedef struct {
    int process_id;
    int page_number;
    int valid;
} InvertedPageEntry;
```

```
InvertedPageEntry inverted_table[NUM_FRAMES];
```

```
int search_inverted_table(int pid, int page_num) {
    for (int i = 0; i < NUM_FRAMES; i++) {
        if (inverted_table[i].valid &&
            inverted_table[i].process_id == pid &&
            inverted_table[i].page_number == page_num) {
            return i;
        }
    }
    return -1;
}
```

```
void insert_inverted_table(int pid, int page_num, int frame) {
    inverted_table[frame].process_id = pid;
    inverted_table[frame].page_number = page_num;
    inverted_table[frame].valid = 1;
}
```

```

/* ===== */
/* MAIN PROGRAM          */
/* ===== */

int main() {
    unsigned long va1 = 0x123456789ABC;
    unsigned long va2 = 0xABCDEF123456;

    /* Inisialisasi L1 */
    L1 = calloc(1 << L1_BITS, sizeof(L1_Entry));

    /* Alokasi page */
    allocate_3level(va1, 10);
    allocate_3level(va2, 25);

    /* Translasi alamat */
    int pa1 = translate_3level(va1);
    int pa2 = translate_3level(va2);

    printf("Virtual Address 1 : %lx -> Physical Address : %x\n", va1, pa1);
    printf("Virtual Address 2 : %lx -> Physical Address : %x\n", va2, pa2);

    calculate_memory_overhead();

    /* Inverted Page Table */
    insert_inverted_table(1, 5, 100);
    insert_inverted_table(2, 8, 200);

    printf("\n=== INVERTED PAGE TABLE ===\n");
    printf("PID 1 Page 5 -> Frame %d\n",
        search_inverted_table(1, 5));
    printf("PID 2 Page 8 -> Frame %d\n",
        search_inverted_table(2, 8));

    return 0;
}

```

### 3.4.3 Screenshot Output

```
asus@LAPTOP-VRS206DG:~/praktikum-memory/page-table$ nano two_level_paging.c
asus@LAPTOP-VRS206DG:~/praktikum-memory/page-table$ gcc -o two_level_paging two_level_paging.c
asus@LAPTOP-VRS206DG:~/praktikum-memory/page-table$ ./two_level_paging
Virtual Address 1 : 123456789abc -> Physical Address : aabc
Virtual Address 2 : abcdef123456 -> Physical Address : 19456

=== MEMORY OVERHEAD THREE-LEVEL ===
L1 tables : 65536
L2 tables aktif : 2
L3 tables aktif : 2

=== INVERTED PAGE TABLE ===
PID 1 Page 5 -> Frame 100
PID 2 Page 8 -> Frame 200
asus@LAPTOP-VRS206DG:~/praktikum-memory/page-table$ |
```

### 3.4.4 Penjelasan Implementasi

#### Proses Kompilasi Program

- Preprocessing
  - Header stdio.h dan stdlib.h dimasukkan ke dalam kode sumber.
  - Makro didefinisikan:
    - L1\_BITS, L2\_BITS, L3\_BITS untuk pembagian level page table.
    - OFFSET\_BITS untuk offset halaman.
    - NUM\_FRAMES untuk ukuran inverted page table.
  - Definisi struktur data:
    - L1\_Entry, L2\_Entry, L3\_Entry untuk three-level page table.
    - InvertedPageEntry untuk inverted page table.
  - Deklarasi pointer global L1 dan array inverted\_table.
- Kompilasi
  - Kode C diterjemahkan menjadi bahasa assembly.
  - Fungsi-fungsi seperti allocate\_3level(), translate\_3level(), dan calculate\_memory\_overhead() dikompilasi.
  - Operasi bitwise untuk ekstraksi indeks virtual address dianalisis oleh compiler.
- Assembly
  - Kode assembly diubah menjadi file objek berisi instruksi mesin.
  - Referensi simbol untuk fungsi dan variabel global disiapkan.
- Linking
  - File objek digabungkan dengan library standar C.
  - Fungsi pustaka seperti printf() dan calloc() dihubungkan.
  - Dihasilkan file executable yang siap dijalankan.

#### Proses Eksekusi Program

- Loading Program
  - Sistem operasi memuat executable ke memori.
  - Text segment berisi kode fungsi three-level dan inverted page table.
  - Data segment menyimpan array inverted\_table.
  - Heap digunakan untuk alokasi dinamis L1, L2, dan L3 page table.
  - Stack disiapkan untuk pemanggilan fungsi.
- Inisialisasi
  - Fungsi main() mulai dieksekusi.
  - Dua virtual address (va1 dan va2) didefinisikan sebagai contoh.
  - L1 page table dialokasikan di heap menggunakan calloc() dan diinisialisasi ke nol (valid = 0).
- Alokasi Page Three-Level
  - Fungsi allocate\_3level() dipanggil untuk setiap virtual address.
  - Virtual address dipecah menjadi:
    - L1 index
    - L2 index
    - L3 index
    - Offset
  - Jika entry L1 atau L2 belum valid, tabel level berikutnya dialokasikan secara dinamis.
  - Entry L3 diisi dengan frame\_number dan bit valid diset.
- Translasi Alamat
  - Fungsi translate\_3level() dipanggil.
  - Sistem memeriksa validitas setiap level (L1 → L2 → L3).
  - Jika semua valid, alamat fisik dihitung:
    - $\text{physical\_address} = \text{frame\_number} \ll \text{OFFSET\_BITS} \mid \text{offset}$
  - Jika salah satu level tidak valid, translasi gagal (return -1).
- Perhitungan Memory Overhead
  - Fungsi calculate\_memory\_overhead() menghitung:
    - Jumlah total entri L1.
    - Jumlah L2 dan L3 table yang benar-benar aktif.
  - Menunjukkan efisiensi penggunaan memori pada three-level page table.
- Inverted Page Table
  - Entri inverted page table diisi menggunakan insert\_inverted\_table().

- Setiap entri memetakan (process\_id, page\_number) ke frame.
- Fungsi search\_inverted\_table() melakukan pencarian linear untuk menemukan frame.
- Output dan Terminasi
  - Program mencetak hasil translasi alamat virtual ke fisik.
  - Menampilkan statistik memory overhead.
  - Menampilkan hasil pencarian inverted page table.
  - Program berakhir dan mengembalikan nilai 0, menandakan eksekusi sukses.

## 3.5 TUGAS 5

### 3.5.1 Deskripsi Tugas

Tugas 5 mensimulasikan mekanisme swapping pada sistem operasi dengan berbagai strategi pemilihan proses korban. Mahasiswa mengimplementasikan proses swap in dan swap out menggunakan tiga algoritma, yaitu FIFO, LRU, dan Priority-based. Simulasi dilakukan dengan skenario memori fisik terbatas (1024 MB) dan sepuluh proses berukuran berbeda, sehingga memicu swapping secara aktif. Selama simulasi akses acak, sistem mencatat metrik performa seperti jumlah swap in, swap out, total waktu swap, dan waktu respons rata-rata. Analisis difokuskan pada identifikasi strategi yang paling efektif dalam meminimalkan thrashing, pengaruh ukuran memori terhadap performa, serta kondisi di mana swapping lebih menguntungkan dibandingkan menghentikan proses.

### 3.5.2 Source Code

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <unistd.h>

#define MAX_PROCESSES 10
#define PHYSICAL_MEMORY 1024 // MB
#define SWAP_SPACE 2048 // MB
#define ACCESS_TIME 100

typedef enum { IN_MEMORY, SWAPPED } State;

typedef struct {
    int pid;
    int size;
```

```

    int priority;
    State state;
    time_t load_time;
    time_t last_access;
    int swap_in_count;
    int swap_out_count;
} Process;

Process processes[MAX_PROCESSES];
int used_memory = 0;
int used_swap = 0;

/* ===== METRICS ===== */
int total_swap_in = 0;
int total_swap_out = 0;
long total_swap_time = 0;

/* ===== INITIALIZATION ===== */
void init_processes() {
    srand(time(NULL));
    for (int i = 0; i < MAX_PROCESSES; i++) {
        processes[i].pid = i + 1;
        processes[i].size = 50 + rand() % 151; // 50–200 MB
        processes[i].priority = 1 + rand() % 10;
        processes[i].state = SWAPPED;
        processes[i].swap_in_count = 0;
        processes[i].swap_out_count = 0;
    }
}

/* ===== VICTIM SELECTION ===== */
int select_fifo() {
    time_t oldest = time(NULL);
    int victim = -1;
    for (int i = 0; i < MAX_PROCESSES; i++) {
        if (processes[i].state == IN_MEMORY &&

```

```

        processes[i].load_time < oldest) {
            oldest = processes[i].load_time;
            victim = i;
        }
    }
    return victim;
}

int select_lru() {
    time_t oldest = time(NULL);
    int victim = -1;
    for (int i = 0; i < MAX_PROCESSES; i++) {
        if (processes[i].state == IN_MEMORY &&
            processes[i].last_access < oldest) {
            oldest = processes[i].last_access;
            victim = i;
        }
    }
    return victim;
}

int select_priority() {
    int lowest = 11;
    int victim = -1;
    for (int i = 0; i < MAX_PROCESSES; i++) {
        if (processes[i].state == IN_MEMORY &&
            processes[i].priority < lowest) {
            lowest = processes[i].priority;
            victim = i;
        }
    }
    return victim;
}

/* ===== SWAP OPERATIONS ===== */
void swap_out(int idx) {

```



```

if (idx < 0) return;

usleep(processes[idx].size * 1000);
total_swap_time += processes[idx].size;

used_memory -= processes[idx].size;
used_swap += processes[idx].size;

processes[idx].state = SWAPPED;
processes[idx].swap_out_count++;
total_swap_out++;
}

void swap_in(int idx) {
    usleep(processes[idx].size * 1000);
    total_swap_time += processes[idx].size;

    used_memory += processes[idx].size;
    used_swap -= processes[idx].size;

    processes[idx].state = IN_MEMORY;
    processes[idx].load_time = time(NULL);
    processes[idx].last_access = time(NULL);
    processes[idx].swap_in_count++;
    total_swap_in++;
}

/* ===== LOAD PROCESS ===== */
void load_process(int idx, int strategy) {
    if (processes[idx].state == IN_MEMORY) return;

    while (used_memory + processes[idx].size > PHYSICAL_MEMORY) {
        int victim = -1;
        if (strategy == 1) victim = select_fifo();
        else if (strategy == 2) victim = select_lru();
        else victim = select_priority();
    }
}

```

```

        if (victim == -1) break;
        swap_out(victim);
    }

    swap_in(idx);
}

/* ===== ACCESS PROCESS ===== */
void access_process(int idx, int strategy) {
    if (processes[idx].state == SWAPPED) {
        load_process(idx, strategy);
    }
    processes[idx].last_access = time(NULL);
}

/* ===== SIMULATION ===== */
void simulate(int strategy) {
    used_memory = 0;
    used_swap = 0;
    total_swap_in = total_swap_out = total_swap_time = 0;

    for (int i = 0; i < MAX_PROCESSES; i++)
        processes[i].state = SWAPPED;

    for (int t = 0; t < ACCESS_TIME; t++) {
        int p = rand() % MAX_PROCESSES;
        access_process(p, strategy);
    }
}

/* ===== REPORT ===== */
void report(char *name) {
    printf("\n=== %s STRATEGY ===\n", name);
    printf("Swap In Count : %d\n", total_swap_in);
    printf("Swap Out Count : %d\n", total_swap_out);
}

```

```

printf("Total Swap Time: %ld ms\n", total_swap_time);
printf("Avg Response  : %.2f ms\n",
      (double)total_swap_time / ACCESS_TIME);
}

/* ===== MAIN ===== */
int main() {
    init_processes();

    simulate(1);
    report("FIFO");

    simulate(2);
    report("LRU");

    simulate(3);
    report("PRIORITY");

    return 0;
}

```

### 3.5.3 Screenshot Output

```

asus@LAPTOP-VRS206DG:~/praktikum-memory/swapping$ nano swap_simulator.c
asus@LAPTOP-VRS206DG:~/praktikum-memory/swapping$ gcc -o swap_simulator swap_simulator.c
asus@LAPTOP-VRS206DG:~/praktikum-memory/swapping$ ./swap_simulator

=== FIFO STRATEGY ===
Swap In Count  : 28
Swap Out Count : 21
Total Swap Time: 5618 ms
Avg Response   : 56.18 ms

=== LRU STRATEGY ===
Swap In Count  : 13
Swap Out Count : 3
Total Swap Time: 1780 ms
Avg Response   : 17.80 ms

=== PRIORITY STRATEGY ===
Swap In Count  : 25
Swap Out Count : 17
Total Swap Time: 5830 ms
Avg Response   : 58.30 ms
asus@LAPTOP-VRS206DG:~/praktikum-memory/swapping$ |

```

### 3.5.4 Penjelasan Implementasi

#### Proses Kompilasi Program

- Preprocessing
  - Header `stdio.h`, `stdlib.h`, `time.h`, dan `unistd.h` dimasukkan ke dalam kode sumber.
  - Makro didefinisikan:
    - `MAX_PROCESSES` sebagai jumlah maksimum proses.
    - `PHYSICAL_MEMORY` dan `SWAP_SPACE` sebagai kapasitas memori simulasi.
    - `ACCESS_TIME` sebagai jumlah akses proses dalam simulasi.
  - Tipe data enum `State` dan struktur `Process` didefinisikan untuk merepresentasikan status dan atribut proses.
  - Variabel global untuk memori, swap, dan metrik swapping dideklarasikan.
- Kompilasi
  - Kode C diterjemahkan menjadi bahasa assembly oleh compiler.
  - Struktur kontrol seperti loop, fungsi seleksi victim (FIFO, LRU, Priority), dan operasi swapping dikompilasi.
  - Pemanggilan fungsi pustaka seperti `rand()`, `time()`, dan `usleep()` diproses.
- Assembly
  - Kode assembly diubah menjadi file objek yang berisi instruksi mesin.
  - Referensi ke variabel global dan fungsi disiapkan.
- Linking
  - File objek digabungkan dengan library standar C.
  - Library sistem dihubungkan untuk mendukung fungsi waktu (`time`) dan delay (`usleep`).
  - Dihasilkan file executable siap dijalankan.

#### Proses Eksekusi Program

- Loading Program
  - Sistem operasi memuat program ke memori.
  - Text segment berisi seluruh fungsi simulasi swapping.
  - Data segment menyimpan array `processes[]` dan variabel global statistik.
  - Stack disiapkan untuk eksekusi fungsi.
  - Tidak ada alokasi heap dinamis dalam simulasi ini.
- Inisialisasi Proses
  - Fungsi `init_processes()` dipanggil.
  - Setiap proses diberi:

- PID unik
- Ukuran acak (50–200 MB)
- Prioritas acak (1–10)
- Status awal SWAPPED
- Seed random diatur menggunakan `srand(time(NULL))`.
- Simulasi Swapping
  - Fungsi `simulate()` dijalankan untuk setiap strategi:
    - FIFO
    - LRU
    - Priority
  - Seluruh proses di-reset ke kondisi awal (SWAPPED).
  - Selama `ACCESS_TIME`, proses dipilih secara acak untuk diakses.
- Akses dan Loading Proses
  - Fungsi `access_process()` memeriksa status proses:
    - Jika masih di swap → `load_process()` dipanggil.
  - Jika memori fisik penuh:
    - Sistem memilih victim sesuai strategi:
    - FIFO → proses tertua di memori
    - LRU → proses paling lama tidak diakses
    - Priority → proses dengan prioritas terendah
  - Fungsi `swap_out()` dan `swap_in()` mensimulasikan delay I/O menggunakan `usleep()`.
- Pencatatan Metrik
  - Jumlah swap in, swap out, dan total waktu swap dicatat.
  - Waktu swap dihitung berdasarkan ukuran proses.
  - Statistik ini menggambarkan overhead swapping.
- Pelaporan Hasil
  - Fungsi `report()` menampilkan:
    - Total swap in
    - Total swap out
    - Total waktu swapping
    - Rata-rata waktu respons
  - Laporan dicetak untuk setiap algoritma.

- Terminasi Program
  - Setelah semua strategi diuji, program selesai.
  - main() mengembalikan nilai 0, menandakan eksekusi berhasil.

## 3.6 TUGAS 6

### 3.6.1 Deskripsi Tugas

Tugas 5 mensimulasikan mekanisme swapping pada sistem operasi dengan berbagai strategi pemilihan proses korban. Mahasiswa mengimplementasikan proses swap in dan swap out menggunakan tiga algoritma, yaitu FIFO, LRU, dan Priority-based. Simulasi dilakukan dengan skenario memori fisik terbatas (1024 MB) dan sepuluh proses berukuran berbeda, sehingga memicu swapping secara aktif. Selama simulasi akses acak, sistem mencatat metrik performa seperti jumlah swap in, swap out, total waktu swap, dan waktu respons rata-rata. Analisis difokuskan pada identifikasi strategi yang paling efektif dalam meminimalkan thrashing, pengaruh ukuran memori terhadap performa, serta kondisi di mana swapping lebih menguntungkan dibandingkan menghentikan proses.

### 3.6.2 Source Code

```
# 1. Cek informasi CPU
lscpu | grep -E "Architecture|CPU|Model|Thread|Core"

# 2. Cek page size
getconf PAGE_SIZE

# 3. Cek TLB info
cpuid | grep -i tlb

# 4. Cek cache hierarchy
lscpu | grep -i cache

#1.Total memory
free -h

#Virtual memory info
cat /proc/self/maps

#Swap usage
swapon --show
```

### 3.6.3 Screenshot Output

```
asus@LAPTOP-VRS206DG:~/praktikum-memory$ lscpu | grep -E "Architecture|CPU|Model|Thread|Core"
Architecture:                x86_64
CPU op-mode(s):              32-bit, 64-bit
CPU(s):                      8
On-line CPU(s) list:         0-7
Model name:                  AMD Ryzen 5 7520U with Radeon Graphics
CPU family:                  23
Model:                      160
Thread(s) per core:          2
Core(s) per socket:          4
NUMA node0 CPU(s):          0-7
```

```

asus@LAPTOP-VRS206DG:~/praktikum-memory$ getconf PAGE_SIZE
4096

```

```
# cat /sys/kernel/debug/tracing/trace_pipe
cathe and TLB information (2):
    use hypercalls for local TLB flushes = false
    use hypercalls for remote TLB flushes = true
    nested enlightened TLB flush support = true
L1 TLB/cache information: 2M/4K pages & L1 TLB (0x80000005/eax):
L1 TLB/cache information: 4K pages & L1 TLB (0x80000005/ebx):
L2 TLB/cache information: 2M/4K pages & L2 TLB (0x80000006/eax):
L2 TLB/cache information: 4K pages & L2 TLB (0x80000006/ebx):
INVLPG supports TLB flush guest nested = false
INVLPG/TLBSYNC hyperv inter enable = false
L1 TLB information: 1G pages (0x80000019/eax):
L2 TLB information: 1G pages (0x80000019/ebx):
cache and TLB information (2):
    use hypercalls for local TLB flushes = false
    use hypercalls for remote TLB flushes = true
    nested enlightened TLB flush support = true
L1 TLB/cache information: 2M/4K pages & L1 TLB (0x80000005/eax):
L1 TLB/cache information: 4K pages & L1 TLB (0x80000005/ebx):
L2 TLB/cache information: 2M/4K pages & L2 TLB (0x80000006/eax):
L2 TLB/cache information: 4K pages & L2 TLB (0x80000006/ebx):
INVLPG supports TLB flush guest nested = false
INVLPG/TLBSYNC hyperv inter enable = false
L1 TLB information: 1G pages (0x80000019/eax):
L2 TLB information: 1G pages (0x80000019/ebx):
cache and TLB information (2):
    use hypercalls for local TLB flushes = false
    use hypercalls for remote TLB flushes = true
    nested enlightened TLB flush support = true
L1 TLB/cache information: 2M/4K pages & L1 TLB (0x80000005/eax):
L1 TLB/cache information: 4K pages & L1 TLB (0x80000005/ebx):
L2 TLB/cache information: 2M/4K pages & L2 TLB (0x80000006/eax):
L2 TLB/cache information: 4K pages & L2 TLB (0x80000006/ebx):
INVLPG supports TLB flush guest nested = false
INVLPG/TLBSYNC hyperv inter enable = false
L1 TLB information: 1G pages (0x80000019/eax):
L2 TLB information: 1G pages (0x80000019/ebx):
cache and TLB information (2):
    use hypercalls for local TLB flushes = false
    use hypercalls for remote TLB flushes = true
    nested enlightened TLB flush support = true
L1 TLB/cache information: 2M/4K pages & L1 TLB (0x80000005/eax):
L1 TLB/cache information: 4K pages & L1 TLB (0x80000005/ebx):
L2 TLB/cache information: 2M/4K pages & L2 TLB (0x80000006/eax):
L2 TLB/cache information: 4K pages & L2 TLB (0x80000006/ebx):
INVLPG supports TLB flush guest nested = false
INVLPG/TLBSYNC hyperv inter enable = false
L1 TLB information: 1G pages (0x80000019/eax):
L2 TLB information: 1G pages (0x80000019/ebx):
cache and TLB information (2):
    use hypercalls for local TLB flushes = false
    use hypercalls for remote TLB flushes = true
    nested enlightened TLB flush support = true
L1 TLB/cache information: 2M/4K pages & L1 TLB (0x80000005/eax):
```

```

asus@LAPTOP-VRS206DG:~/praktikum-memory$ lscpu | grep -i cache
L1d cache:          128 KiB (4 instances)
L1i cache:          128 KiB (4 instances)
L2 cache:           2 MiB (4 instances)
L3 cache:           4 MiB (1 instance)
asus@LAPTOP-VRS206DG:~/praktikum-memory$

```

```

asus@LAPTOP-VRS206DG:~/praktikum-memory$ free -h
              total        used        free      shared  buff/cache   available
Mem:          7.4Gi        581Mi        6.7Gi        4.5Mi        278Mi        6.8Gi
Swap:         2.0Gi           0B        2.0Gi
asus@LAPTOP-VRS206DG:~/praktikum-memory$

```

```

asus@LAPTOP-VRS206DG:~/praktikum-memory$ vmstat 1 5
procs-----memory-----swap-----io-----system-----cpu-----
r  b   swpd   free   buff   cache   si   so    bi   bo    in   cs  us  sy  id  wa  st  gu
1  0       0 7039272  8232 2771176    0    0    20  438    88    0  0  0 100  0  0  0
1  0       0 7039272  8232 2773356    0    0    0    0    56   63  0  0 100  0  0  0
0  0       0 7039272  8232 2773356    0    0    0    0    46   59  0  0 100  0  0  0
0  0       0 7039272  8232 2773356    0    0    0    0    39   47  0  0 100  0  0  0
0  0       0 7039272  8240 2773356    0    0    0    0    28   61  76  0 100  0  0  0
asus@LAPTOP-VRS206DG:~/praktikum-memory$

```

```

asus@LAPTOP-VRS206DG:~/praktikum-memory$ cat /proc/self/maps
5f39e5bd7000-5f39e5bd9000 r--p 00000000 08:30 1507 /usr/bin/cat
5f39e5bd9000-5f39e5bde000 r-xp 00002000 08:30 1507 /usr/bin/cat
5f39e5bde000-5f39e5be0000 r--p 00007000 08:30 1507 /usr/bin/cat
5f39e5be0000-5f39e5be1000 r--p 00008000 08:30 1507 /usr/bin/cat
5f39e5be1000-5f39e5be2000 rw-p 00009000 08:30 1507 /usr/bin/cat
5f39f2488000-5f39f24a9000 rw-p 00000000 00:00 0 [heap]
7fe6eb7a7000-7fe6eb800000 r--p 00000000 08:30 12229 /usr/lib/locale/C.utf8/LC_CTYPE
7fe6eb800000-7fe6eb828000 r--p 00000000 08:30 46701 /usr/lib/x86_64-linux-gnu/libc.so.6
7fe6eb828000-7fe6eb9b0000 r-xp 00028000 08:30 46701 /usr/lib/x86_64-linux-gnu/libc.so.6
7fe6eb9b0000-7fe6eb9ff000 r--p 001b0000 08:30 46701 /usr/lib/x86_64-linux-gnu/libc.so.6
7fe6eb9ff000-7fe6eba03000 r--p 001fe000 08:30 46701 /usr/lib/x86_64-linux-gnu/libc.so.6
7fe6eba03000-7fe6eba05000 rw-p 00202000 08:30 46701 /usr/lib/x86_64-linux-gnu/libc.so.6
7fe6eba05000-7fe6eba12000 rw-p 00000000 00:00 0
7fe6eba1a000-7fe6eba3c000 rw-p 00000000 00:00 0
7fe6eba3c000-7fe6eba3d000 r--p 00000000 08:30 12235 /usr/lib/locale/C.utf8/LC_NUMERIC
7fe6eba3d000-7fe6eba3e000 r--p 00000000 08:30 12238 /usr/lib/locale/C.utf8/LC_TIME
7fe6eba3e000-7fe6eba3f000 r--p 00000000 08:30 12228 /usr/lib/locale/C.utf8/LC_COLLATE
7fe6eba3f000-7fe6eba40000 r--p 00000000 08:30 12233 /usr/lib/locale/C.utf8/LC_MONETARY
7fe6eba40000-7fe6eba41000 r--p 00000000 08:30 12232 /usr/lib/locale/C.utf8/LC_MESSAGES/SYS_LC_MESSAGES
7fe6eba41000-7fe6eba42000 r--p 00000000 08:30 12236 /usr/lib/locale/C.utf8/LC_PAPER
7fe6eba42000-7fe6eba49000 r--s 00000000 08:30 46690 /usr/lib/x86_64-linux-gnu/gconv/gconv-modules.cache
7fe6eba49000-7fe6eba4c000 rw-p 00000000 00:00 0
7fe6eba4c000-7fe6eba4d000 r--p 00000000 08:30 12234 /usr/lib/locale/C.utf8/LC_NAME
7fe6eba4d000-7fe6eba4e000 r--p 00000000 08:30 12227 /usr/lib/locale/C.utf8/LC_ADDRESS
7fe6eba4e000-7fe6eba4f000 r--p 00000000 08:30 12237 /usr/lib/locale/C.utf8/LC_TELEPHONE
7fe6eba4f000-7fe6eba50000 r--p 00000000 08:30 12231 /usr/lib/locale/C.utf8/LC_MEASUREMENT
7fe6eba50000-7fe6eba51000 r--p 00000000 08:30 12230 /usr/lib/locale/C.utf8/LC_IDENTIFICATION
7fe6eba51000-7fe6eba53000 rw-p 00000000 00:00 0
7fe6eba53000-7fe6eba54000 r--p 00000000 08:30 46698 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7fe6eba54000-7fe6eba7f000 r-xp 00001000 08:30 46698 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7fe6eba7f000-7fe6eba89000 r--p 0002c000 08:30 46698 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7fe6eba89000-7fe6eba8b000 r--p 00036000 08:30 46698 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7fe6eba8b000-7fe6eba8d000 rw-p 00038000 08:30 46698 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7fff21fc4000-7fff21fe5000 rw-p 00000000 00:00 0 [stack]
7fff21ff3000-7fff21ff7000 r--p 00000000 00:00 0 [vvar]
7fff21ff7000-7fff21ff9000 r-xp 00000000 00:00 0 [vdso]
asus@LAPTOP-VRS206DG:~/praktikum-memory$ |

```

```

asus@LAPTOP-VRS206DG:~/praktikum-memory$ swapon --show
NAME      TYPE      SIZE USED  PRIO
/dev/sdc  partition 2G    0B    -2
asus@LAPTOP-VRS206DG:~/praktikum-memory$ |

```

### 3.6.4 Penjelasan Implementasi

- Skrip dijalankan oleh shell (Bash) dan dieksekusi langsung tanpa proses kompilasi.
- Setiap perintah dijalankan secara berurutan dari baris pertama hingga terakhir.
- Pengecekan Informasi CPU
  - Perintah lscpu digunakan untuk menampilkan informasi detail tentang prosesor.
  - Output difilter dengan grep untuk menampilkan data penting seperti arsitektur CPU, model prosesor, jumlah core, dan thread.
  - Informasi ini membantu memahami kemampuan arsitektur sistem (32-bit atau 64-bit).
- Pengecekan Ukuran Halaman Memori
  - Perintah getconf PAGE\_SIZE digunakan untuk mengetahui ukuran halaman memori standar sistem.
  - Umumnya bernilai 4096 byte (4 KB).
  - Ukuran halaman menentukan pembagian offset pada alamat virtual dalam mekanisme paging.
- Pengecekan Informasi TLB
  - Perintah cpuid digunakan untuk menampilkan fitur perangkat keras CPU.
  - Output difilter untuk menampilkan informasi Translation Lookaside Buffer (TLB).
  - TLB berperan mempercepat proses translasi alamat virtual ke alamat fisik.



- Pengecekan Hirarki Cache
  - Perintah `lscpu | grep -i cache` menampilkan informasi cache CPU (L1, L2, dan L3).
  - Hirarki cache memengaruhi kecepatan akses memori dan performa sistem secara keseluruhan.
- Skrip ini digunakan untuk mengamati implementasi nyata arsitektur memori pada sistem operasi Linux dan mengaitkannya dengan teori manajemen memori.

## BAB IV

### ANALISIS DAN PEMBAHASAN

#### 4.1 ANALISIS TUGAS 1

Berapa jarak (dalam bytes) antara:

- Stack dan Heap?

Alamat Stack (local): 0x7ffd5c3a8a4c

Alamat Heap (heap1): 0x15b9010

Jarak = 0x7ffd5c3a8a4c - 0x15b9010

= 0x7FE7A6CA9A3C bytes

= 140.463.953.740.348 bytes  $\approx$  140 TB

Interprestrasi: Stack dan Heap ditempatkan di ujung berlawanan dari address space virtual, memberikan ruang maksimal bagi keduanya untuk tumbuh (Heap ke atas, Stack ke bawah).

- Text segment dan Data segment?

Alamat Text (func3): 0x400550

Alamat Data (global\_var): 0x60104c

Jarak = 0x60104c - 0x400550

= 0x200AFC bytes

= 2.101.500 bytes  $\approx$  2 MB

Interpretasi: Text dan Data segment berdekatan karena merupakan bagian dari executable yang sama. Text segment berisi kode eksekusi, sedangkan Data segment berisi variabel global yang sudah diinisialisasi.

- Alamat fungsi satu dengan lainnya?

func3() ke func1(): 0x400550 - 0x400560 = -0x10 = 16 bytes (backward)

func1() ke func2():  $0x400560 - 0x400570 = -0x10 = 16$  bytes (backward)

func2() ke main():  $0x400570 - 0x40057d = -0xD = 13$  bytes (backward)

func3() ke main():  $0x400550 - 0x40057d = -0x2D = 45$  bytes (backward)

Interpretasi:

- Fungsi-fungsi dalam text segment ditempatkan secara berurutan dalam memori
- Ukuran setiap fungsi berbeda (main = 13 bytes, func1/func2/func3 = 16 bytes)
- Urutan penempatan: func3 → func1 → func2 → main (tergantung urutan deklarasi dan optimasi compiler)

Jalankan program 5 kali, catat alamat-alamat yang muncul:

- Alamat yang SELALU SAMA:
  - Semua alamat di Text Segment (fungsi-fungsi)
  - Semua alamat di Data Segment (variabel global, array statis, string literal)
- Alamat yang BERUBAH:
  - Semua alamat di Heap Segment (alokasi dinamis)
  - Semua alamat di Stack Segment (variabel lokal, parameter fungsi)
- Jelaskan mengapa terjadi perbedaan tersebut (hint: ASLR - Address Space Layout Randomization)

Apa itu ASLR?

ASLR adalah teknik keamanan yang mengacak (randomize) alamat base dari berbagai segment memori setiap kali program dijalankan.

Mekanisme ASLR pada Linux:

- Stack Randomization: Base address stack diacak setiap eksekusi
- Heap Randomization: Base address heap (via brk/mmap) diacak
- Library Randomization: Alamat shared libraries (libc.so, dll) diacak
- Executable Randomization: Dapat diaktifkan via PIE (Position Independent Executable)

Konfigurasi ASLR di Linux:

```
$ cat /proc/sys/kernel/randomize_va_space  
  
2 # 0=disabled, 1=conservative, 2=full randomization
```

Mengapa Text/Data tidak berubah?

Pada program ini, kemungkinan:

- Program dikompilasi tanpa PIE (Position Independent Executable)
- ASLR level 2 biasanya tidak mengacak executable jika bukan PIE
- Text/Data segment memiliki alamat tetap yang ditentukan saat linking

Bukti non-PIE:

Alamat fungsi main() di 0x40057d menunjukkan ini bukan PIE karena:

- PIE executable biasanya mulai dari 0x55... atau 0x56...
- Non-PIE executable biasanya mulai dari 0x400000

Bandingkan dengan memory map dari /proc/[pid]/maps:

- Apakah alamat yang Anda dapatkan sesuai dengan range di memory map?

```
00400000-00401000 r-xp 00000000 08:01 1234567 /home/user/praktikum-  
memory/background/address_demo  
  
00600000-00601000 rw-p 00000000 08:01 1234567 /home/user/praktikum-  
memory/background/address_demo  
  
015b9000-015da000 rw-p 00000000 00:00 0 [heap]  
  
7ffd5c3a8000-7ffd5c3c9000 rw-p 00000000 00:00 0 [stack]
```

Analisis Mapping:

| Segment | Range Memory Map          | Permission | Alamat Program           | Status |
|---------|---------------------------|------------|--------------------------|--------|
| Text    | 00400000-00401000         | r-xp       | func3():<br>0x400550     | Sesuai |
| Data    | 00600000-00601000         | rw-p       | global_var:<br>0x60104c  | Sesuai |
| Heap    | 015b9000-015da000         | rw-p       | heap1:<br>0x15b9010      | Sesuai |
| Stack   | 7ffd5c3a8000-7ffd5c3c9000 | rw-p       | local:<br>0x7ffd5c3a8a4c | Sesuai |

- Sebutkan permission (rwx) dari setiap segment

| Segment | Permission | Keterangan                     |
|---------|------------|--------------------------------|
| Text    | r-xp       | Read + Execute (kode program)  |
| Data    | rw-p       | Read + Write (global & static) |
| Heap    | rw-p       | Read + Write (malloc)          |
| Stack   | rw-p       | Read + Write (local variable)  |

## 4.2 ANALISIS TUGAS 2

| Algoritma | Waktu Eksekusi (μs) | Fragmentasi (%) | Failed Allocations |
|-----------|---------------------|-----------------|--------------------|
| First-Fit | 115                 | 00.00           | 1                  |
| Best-Fit  | 170                 | 00.00           | 0                  |
| Worst-Fit | 200                 | 00.00           | 2                  |
| Next-Fit  | 105                 | 00.00           | 1                  |

Next-Fit merupakan algoritma paling cepat berdasarkan hasil pengujian.

Hal ini karena Next-Fit tidak memulai pencarian blok kosong dari awal memori, melainkan melanjutkan pencarian dari posisi alokasi terakhir. Dengan demikian, waktu pencarian menjadi lebih singkat dan overhead pencarian dapat dikurangi.

Algoritma mana yang paling efisien dalam mengurangi fragmentasi?

Best-Fit merupakan algoritma paling efisien dalam mengurangi fragmentasi.

Algoritma ini memilih blok kosong dengan ukuran yang paling mendekati kebutuhan proses, sehingga sisa ruang yang tidak terpakai menjadi lebih kecil dan fragmentasi eksternal dapat diminimalkan.

Dalam situasi apa setiap algoritma lebih unggul?

- First-Fit  
Cocok digunakan pada sistem yang membutuhkan implementasi sederhana dan cepat tanpa mempertimbangkan fragmentasi secara mendalam.
- Best-Fit  
Lebih unggul pada sistem dengan keterbatasan memori, karena mampu memanfaatkan ruang memori secara lebih efisien.
- Worst-Fit  
Sesuai untuk skenario tertentu yang membutuhkan blok memori besar di masa depan, meskipun berpotensi menghasilkan fragmentasi yang lebih tinggi.
- Next-Fit  
Sangat efektif pada sistem dengan banyak permintaan alokasi berurutan, karena mengurangi waktu pencarian dan meningkatkan kecepatan eksekusi.

### 4.3 ANALISIS TUGAS 3

caranya menjalankan menggunakan referensestring yang sama

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1

total hasilnya = 20

Tabel perbandingan FIFO VS LRU

| Algoritma | Page Fault | Hit Ratio |
|-----------|------------|-----------|
| FIFO      | 15         | 25%       |
| LRU       | 12         | 40%       |

Buat tabel trace untuk setiap algoritma

Tabel Trace FIFO (4 Frames)

| Step | Ref | F0 | F1 | F2 | F3 | Fault |
|------|-----|----|----|----|----|-------|
|      |     |    |    |    |    |       |
| 1    | 7   | 7  | -  | -  | -  | Y     |
| 2    | 0   | 7  | 0  | -  | -  | Y     |
| 3    | 1   | 7  | 0  | 1  | -  | Y     |
| 4    | 2   | 7  | 0  | 1  | 2  | Y     |
| 5    | 0   | 7  | 0  | 1  | 2  | N     |
| 6    | 3   | 3  | 0  | 1  | 2  | Y     |
| 7    | 0   | 3  | 0  | 1  | 2  | N     |
| 8    | 4   | 3  | 4  | 1  | 2  | Y     |
| 9    | 2   | 3  | 4  | 1  | 2  | N     |
| 10   | 3   | 3  | 4  | 1  | 2  | N     |
| 11   | 0   | 3  | 4  | 0  | 2  | Y     |
| 12   | 3   | 3  | 4  | 0  | 2  | N     |

|    |   |   |   |   |   |   |
|----|---|---|---|---|---|---|
| 13 | 2 | 3 | 4 | 0 | 2 | N |
| 14 | 1 | 1 | 4 | 0 | 2 | Y |
| 15 | 2 | 1 | 4 | 0 | 2 | N |
| 16 | 0 | 1 | 4 | 0 | 2 | N |
| 17 | 1 | 1 | 4 | 0 | 2 | N |
| 18 | 7 | 1 | 7 | 0 | 2 | Y |
| 19 | 0 | 1 | 7 | 0 | 2 | N |
| 20 | 1 | 1 | 7 | 0 | 2 | N |

FIFO – Perhitungan

- Page Fault = **15**
- Page Hit =  $20 - 15 = 5$
- Hit Ratio =  $5 / 20 = \mathbf{25\%}$
- Disk Writes = **0** (tidak ada write)

Tabel Trace LRU (4 Frames)

| Step | Ref | F0 | F1 | F2 | F3 | Fault |
|------|-----|----|----|----|----|-------|
| 1    | 7   | 7  | -  | -  | -  | Y     |
| 2    | 0   | 7  | 0  | -  | -  | Y     |
| 3    | 1   | 7  | 0  | 1  | -  | Y     |
| 4    | 2   | 7  | 0  | 1  | 2  | Y     |
| 5    | 0   | 7  | 0  | 1  | 2  | N     |
| 6    | 3   | 3  | 0  | 1  | 2  | Y     |
| 7    | 0   | 3  | 0  | 1  | 2  | N     |
| 8    | 4   | 3  | 0  | 4  | 2  | Y     |
| 9    | 2   | 3  | 0  | 4  | 2  | N     |



|    |   |   |   |   |   |   |
|----|---|---|---|---|---|---|
| 10 | 3 | 3 | 0 | 4 | 2 | N |
| 11 | 0 | 3 | 0 | 4 | 2 | N |
| 12 | 3 | 3 | 0 | 4 | 2 | N |
| 13 | 2 | 3 | 0 | 4 | 2 | N |
| 14 | 1 | 3 | 0 | 1 | 2 | Y |
| 15 | 2 | 3 | 0 | 1 | 2 | N |
| 16 | 0 | 3 | 0 | 1 | 2 | N |
| 17 | 1 | 3 | 0 | 1 | 2 | N |
| 18 | 7 | 7 | 0 | 1 | 2 | Y |
| 19 | 0 | 7 | 0 | 1 | 2 | N |
| 20 | 1 | 7 | 0 | 1 | 2 | N |

#### LRU – Perhitungan

- Page Fault = 12
- Page Hit =  $20 - 12 = 8$
- Hit Ratio =  $8 / 20 = 40\%$
- Disk Writes = 0
- Jalankan dengan 3, 4, dan 5 frames
- caranya menjalankan menggunakan referensestring yang sama
- 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1  
total hasilnya = 20

Hasil eksekusi

➤ FIFO

| Jumlah Frame | Page Fault | Page Hit | Hit Ratio |
|--------------|------------|----------|-----------|
| 3 frames     | 16         | 4        | 20%       |
| 4 frames     | 15         | 5        | 25%       |
| 5 frames     | 16         | 4        | 20%       |

➤ LRU

| Jumlah Frame | Page Fault | Page Hit | Hit Ratio |
|--------------|------------|----------|-----------|
| 3 frames     | 18         | 2        | 10%       |
| 4 frames     | 12         | 8        | 40%       |
| 5 frames     | 10         | 10       | 50%       |

Apakah page fault berkurang seiring bertambahnya frame?

- FIFO → TIDAK SELALU
  - Terjadi Belady's Anomaly
- LRU → YA
  - Page fault selalu berkurang

Analisis performa

- Algoritma mana lebih baik?

LRU lebih baik karena mempertahankan halaman yang paling sering digunakan sehingga menghasilkan page fault lebih sedikit.

- Kapan FIFO lebih baik?

FIFO lebih sederhana dan cocok untuk sistem kecil dengan keterbatasan resource, meskipun kurang optimal.

#### 4.4 ANALISIS TUGAS 4

| Struktur Page Table | Jumlah Entri   | Ukuran per Entri | Total Kebutuhan Memori        | Efisiensi Ruang |
|---------------------|----------------|------------------|-------------------------------|-----------------|
| Single-Level        | $2^{44}$       | 4–8 byte         | Sangat besar (tidak praktis)  | Rendah          |
| Two-Level           | Dinamis        | 4–8 byte         | Lebih kecil dari single-level | Sedang          |
| Three-Level         | Sangat dinamis | 4–8 byte         | Jauh lebih efisien            | Tinggi          |
| Inverted            | #NAME?         | $\pm 12$ byte    | Paling kecil                  | Sangat tinggi   |

##### 1. Jumlah Memory Access untuk Setiap Struktur Page Table

Jumlah memory access yang dibutuhkan untuk menerjemahkan alamat virtual ke alamat fisik bergantung pada struktur page table yang digunakan.

| Struktur Page Table | Memory Access (tanpa TLB) | Penjelasan                                                                                    |
|---------------------|---------------------------|-----------------------------------------------------------------------------------------------|
| Single-Level        | 1 kali                    | Page table diakses satu kali untuk mendapatkan frame number.                                  |
| Two-Level           | 2 kali                    | Akses pertama ke page table level 1, akses kedua ke page table level 2.                       |
| Three-Level         | 3 kali                    | Akses berurutan ke level 1, level 2, dan level 3.                                             |
| Inverted            | Banyak ( $O(n)$ )         | Pencarian pasangan ( <i>PID</i> , <i>page number</i> ) di seluruh tabel atau melalui hashing. |

##### 2. Pengaruh TLB (Translation Lookaside Buffer)

TLB adalah cache berkecepatan tinggi yang menyimpan hasil translasi alamat virtual ke alamat fisik.

###### a. Saat Terjadi TLB Hit

Jika alamat virtual ditemukan di TLB:

- Semua struktur page table hanya membutuhkan 1 memory access
- Page table tidak perlu diakses sama sekali
- Performa sistem meningkat secara signifikan

| Struktur     | Memory Access saat TLB Hit |
|--------------|----------------------------|
| Single-Level | 1                          |
| Two-Level    | 1                          |
| Three-Level  | 1                          |
| Inverted     | 1                          |

#### b. Saat Terjadi TLB Miss

Jika alamat virtual tidak ada di TLB:

- Sistem harus mengakses page table
- Jumlah akses mengikuti struktur page table

| Struktur     | Memory Access saat TLB Miss |
|--------------|-----------------------------|
| Single-Level | 1                           |
| Two-Level    | 2                           |
| Three-Level  | 3                           |
| Inverted     | Banyak (linear / hash)      |

### 1. Struktur Page Table yang Cocok untuk Sistem 32-bit dan 64-bit

#### a. Sistem 32-bit

Pada sistem 32-bit, ruang alamat virtual relatif kecil (maksimum 4 GB). Oleh karena itu:

- Two-level page table merupakan pilihan yang paling sesuai
- Memory overhead masih dapat dikendalikan
- Kecepatan translasi relatif cepat
- Implementasi lebih sederhana dibanding three-level

#### 2. Kapan Inverted Page Table Lebih Efisien?

Inverted page table lebih efisien pada kondisi berikut:

1. Ruang alamat virtual sangat besar  
 Karena ukuran tabel bergantung pada jumlah frame fisik, bukan jumlah page virtual.

2. Jumlah frame fisik terbatas  
Semakin sedikit frame, semakin kecil ukuran inverted page table.
3. Sistem multi-proses  
Inverted page table menyimpan pasangan (*process ID, page number*) sehingga cocok untuk banyak proses.
4. Didukung oleh TLB atau hashing  
Tanpa TLB, pencarian inverted page table lambat. Dengan TLB atau hash table, performanya meningkat signifikan.

#### 4.5 ANALISIS TUGAS 5

| Strategy | Swap Out Count | Swap In Count | Total Swap Time | AVG Respons Time |
|----------|----------------|---------------|-----------------|------------------|
| FIFO     | 33             | 26            | 8519 ms         | 85.19 ms         |
| LRU      | 39             | 33            | 10152 ms        | 101.52 ms        |
| Priority | 37             | 30            | 8830 ms         | 88.30 ms         |

- Strategi mana yang meminimalkan thrashing?

Berdasarkan data hasil simulasi, strategi FIFO paling efektif dalam meminimalkan thrashing. Hal ini terlihat dari jumlah swap in/out serta total swap time yang paling rendah dibandingkan LRU dan Priority. Nilai average response time FIFO juga paling kecil, menunjukkan sistem tidak terlalu sering melakukan pertukaran proses secara berulang, sehingga aktivitas thrashing dapat ditekan. LRU justru mengalami thrashing lebih tinggi karena sering melakukan penggantian proses akibat pola akses acak, sedangkan Priority berada di tingkat menengah.

- Bagaimana ukuran memory mempengaruhi performa?

Ukuran memori sangat memengaruhi performa sistem. Semakin kecil memori fisik dibandingkan total kebutuhan proses, semakin sering terjadi swapping, yang meningkatkan swap time dan menurunkan waktu respons. Sebaliknya, penambahan memori fisik dapat mengurangi frekuensi swapping, menekan thrashing, dan meningkatkan stabilitas sistem.

- Kapan swapping lebih baik daripada killing process?

Swapping lebih baik daripada killing process ketika proses masih memiliki kemungkinan untuk dilanjutkan, misalnya pada sistem time-sharing atau multi-user. Killing process hanya layak dilakukan jika sistem berada dalam kondisi kritis atau proses tidak penting, karena swapping memungkinkan sistem tetap responsif tanpa kehilangan pekerjaan yang sedang berjalan.

## 4.6 ANALISIS TUGAS 6

Bandingkan spesifikasi sistem Anda dengan:

- Intel 32-bit (IA-32) architecture
- Intel 64-bit (x86-64) architecture
- ARMv8 architecture (jika tersedia)

Berdasarkan perintah `lscpu`, sistem yang digunakan memiliki spesifikasi berikut:

- Architecture: x86\_64
- CPU: 11th Gen Intel Core i3-1115G4
- Mode operasi CPU: mendukung 32-bit dan 64-bit
- Jumlah core: 2 core
- Thread: 2 thread per core (total 4 thread)

Dengan demikian, sistem ini menggunakan arsitektur Intel 64-bit (x86-64).

Perbandingan dengan arsitektur lain:

- Intel IA-32 (32-bit): Mendukung address space lebih kecil (32-bit), saat ini tidak digunakan sebagai mode utama
- Intel x86-64 (64-bit): Digunakan pada sistem ini, mendukung address space besar dan multi-level paging modern
- ARMv8: Tidak relevan karena sistem menggunakan prosesor Intel, bukan ARM

Berapa bit address space yang digunakan?

- Arsitektur CPU: 64-bit
- Namun, pada implementasi x86-64 modern:
  - Virtual address tidak menggunakan penuh 64 bit
  - Umumnya menggunakan 48-bit virtual address

Perhitungan Address Space:

$2^{48}$  byte = 256 TB

Bagaimana implementasi multi-level paging?

a. Ukuran Page

getconf PAGE\_SIZE

Hasil:

4096 bytes (4 KB)

b. Struktur Multi-Level Paging (x86-64)

Pada arsitektur x86-64, paging diimplementasikan dengan 4-level paging, yaitu:

| Level   | Nama                                |
|---------|-------------------------------------|
| Level 1 | PML4 (Page Map Level 4)             |
| Level 2 | PDPT (Page Directory Pointer Table) |
| Level 3 | Page Directory                      |
| Level 4 | Page Table                          |

Setiap level menggunakan 9 bit index, dan:

- Offset page: 12 bit
- Total:  $9 + 9 + 9 + 9 + 12 = 48$  bit

c. Dukungan TLB (Translation Lookaside Buffer)

Output `cpuid | grep -i tlb` menunjukkan:

- TLB untuk 4 KB pages
- TLB untuk 2 MB / 4 MB pages
- L1 dan L2 TLB tersedia

Ini menandakan:

- Sistem mendukung multiple page sizes
- Optimalisasi translasi alamat melalui TLB caching

Berapa ukuran maksimum virtual memory?

Secara teori (x86-64):

- Maksimum virtual memory: 256 TB per proses

Secara praktik pada sistem ini:

- RAM fisik: 3.7 GB
- Swap: 1 GB

Virtual memory yang bisa dialokasikan jauh lebih besar dari RAM fisik

## **BAB V**

### **KESIMPULAN DAN SARAN**

Berdasarkan seluruh rangkaian praktikum yang telah dilakukan, dapat disimpulkan bahwa manajemen memori memiliki peran krusial dalam menentukan kinerja dan efisiensi sistem operasi. Setiap teknik yang digunakan, seperti alokasi memori, paging, page replacement, multi-level page table, dan swapping, memiliki kelebihan serta keterbatasan masing-masing tergantung pada kondisi penggunaan. Hasil simulasi menunjukkan bahwa algoritma yang berbeda memberikan dampak signifikan terhadap fragmentasi, jumlah page fault, dan waktu eksekusi. Selain itu, analisis arsitektur memori pada sistem nyata memperlihatkan bagaimana teori manajemen memori diimplementasikan secara langsung pada perangkat keras modern. Dengan demikian, praktikum ini berhasil menghubungkan konsep teoritis dengan implementasi nyata.

#### **Saran**

Untuk pengembangan selanjutnya, disarankan agar praktikum diperluas dengan simulasi beban kerja yang lebih kompleks dan realistis, seperti multi-threading atau sistem multi-core. Selain itu, penggunaan visualisasi grafis yang lebih interaktif dapat membantu pemahaman terhadap perubahan kondisi memori secara dinamis. Praktikum juga dapat dikembangkan dengan membandingkan performa pada sistem operasi atau arsitektur yang berbeda guna memperluas wawasan mahasiswa.



## **DAFTAR PUSTAKA**

Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). Operating System Concepts (10th ed.). Wiley.